



By : Anas Khamees

Embedded C

Exploring the Embedded Programming.



1. Role of Embedded C in Embedded Systems

Embedded C is a variation of the standard C language **customized for programming microcontrollers** and low-level hardware.

Why use Embedded C?

- **Low-level hardware access:** You can control registers, ports, pins directly
- **Fast and efficient:** Runs on devices with very limited resources (e.g., 2 KB RAM)
- **Widely supported:** All major MCU vendors (e.g., STM32, AVR, PIC) support Embedded C



2. Difference Between C for PC vs Embedded C

Feature	C for PC	Embedded C
Platform	General-purpose OS (Windows/Linux)	Bare-metal (no OS) or RTOS on microcontrollers
I/O	Uses stdin, stdout, files	Uses GPIO, UART, ADC, etc.
Memory	Plenty of RAM & disk	Very limited memory (KBs)
Execution Model	Multiple processes, threads, OS calls	Runs in loops, often without OS
Hardware Access	Indirect (OS-managed)	Direct register manipulation
Typical Use Case	Apps, scripts, games, websites	Sensors, control systems, real-time devices



3. The Embedded Toolchain

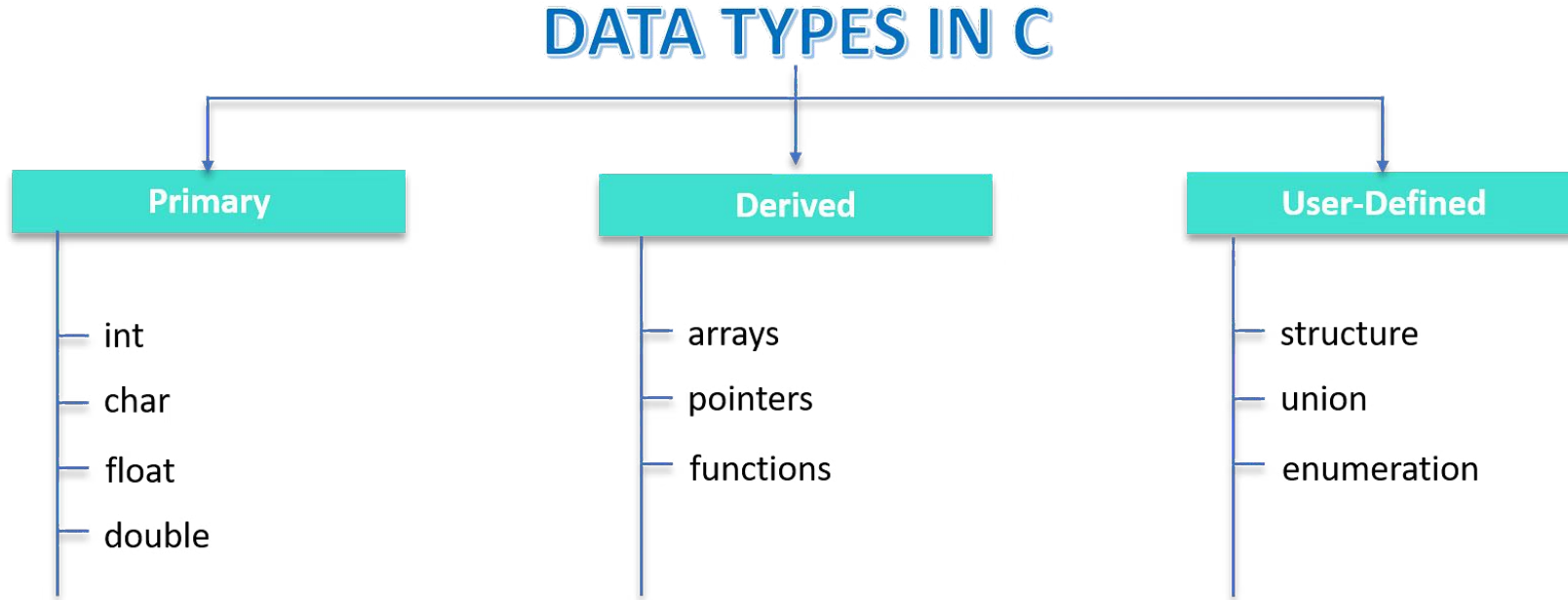
To compile and run Embedded C on microcontrollers, we use a **toolchain** — a set of tools that converts your code into a binary file your MCU can understand.

Tool	Purpose	Example Tools
Editor / IDE	Write your code	VS code IDE, Eclipse, STM32CubeIDE
Compiler	Converts C to machine code	avr-gcc, arm-gcc, MPLAB XC8
Assembler (optional)	Converts assembly to binary	as (GNU assembler)
Linker	Combines compiled files into final output	ld (GNU linker)
Debugger	Step through code, set breakpoints	Atmel Studio, Keil, GDB, OpenOCD
Programmer / Flasher	Loads binary to MCU	USBasp, ST-Link, J-Link



Topic 1: Data Types (Memory Size & HW Relevance)

- Standard data types: int, char, float, double
- Memory size variations (int may be 16-bit or 32-bit based on MCU)
- Importance of choosing correct type for **memory-constrained systems**



Type	Storage size	Value range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	8 bytes	-9223372036854775808 to 9223372036854775807
unsigned long	8 bytes	0 to 18446744073709551615



```

#include <stdio.h>
#include <limits.h>

int main(int argc, char** argv) {

    printf("CHAR_BIT      :   %d\n", CHAR_BIT);
    printf("CHAR_MAX       :   %d\n", CHAR_MAX);
    printf("CHAR_MIN       :   %d\n", CHAR_MIN);
    printf("INT_MAX        :   %d\n", INT_MAX);
    printf("INT_MIN        :   %d\n", INT_MIN);
    printf("LONG_MAX       :   %ld\n", (long) LONG_MAX);
    printf("LONG_MIN       :   %ld\n", (long) LONG_MIN);
    printf("SCHAR_MAX      :   %d\n", SCHAR_MAX);
    printf("SCHAR_MIN      :   %d\n", SCHAR_MIN);
    printf("SHRT_MAX       :   %d\n", SHRT_MAX);
    printf("SHRT_MIN       :   %d\n", SHRT_MIN);
    printf("UCHAR_MAX      :   %d\n", UCHAR_MAX);
    printf("UINT_MAX       :   %u\n", (unsigned int) UINT_MAX);
    printf("ULONG_MAX      :   %lu\n", (unsigned long) ULONG_MAX);
    printf("USHRT_MAX      :   %d\n", (unsigned short) USHRT_MAX);

    return 0;
}

```

```

CHAR_BIT      :   8
CHAR_MAX       :   127
CHAR_MIN       :   -128
INT_MAX        :   2147483647
INT_MIN        :   -2147483648
LONG_MAX       :   9223372036854775807
LONG_MIN       :   -9223372036854775808
SCHAR_MAX      :   127
SCHAR_MIN      :   -128
SHRT_MAX       :   32767
SHRT_MIN       :   -32768
UCHAR_MAX      :   255
UINT_MAX       :   4294967295
ULONG_MAX      :   18446744073709551615
USHRT_MAX      :   65535

```



Type	Storage size	Value range	Precision
float	4 byte	1.2E-38 to 3.4E+38	6 decimal places
double	8 byte	2.3E-308 to 1.7E+308	15 decimal places
long double	10 byte	3.4E-4932 to 1.1E+4932	19 decimal places

```
#include <stdio.h>
#include <float.h>

int main(int argc, char** argv) {

    printf("Storage size for float : %zu \n", sizeof(float));
    printf("FLT_MAX      : %g\n", (float) FLT_MAX);
    printf("FLT_MIN      : %g\n", (float) FLT_MIN);
    printf("-FLT_MAX     : %g\n", (float) -FLT_MAX);
    printf("-FLT_MIN     : %g\n", (float) -FLT_MIN);
    printf("DBL_MAX      : %g\n", (double) DBL_MAX);
    printf("DBL_MIN      : %g\n", (double) DBL_MIN);
    printf("-DBL_MAX     : %g\n", (double) -DBL_MAX);
    printf("Precision value: %d\n", FLT_DIG );

    return 0;
}
```

```
Storage size for float : 4
FLT_MAX      : 3.40282e+38
FLT_MIN      : 1.17549e-38
-FLT_MAX     : -3.40282e+38
-FLT_MIN     : -1.17549e-38
DBL_MAX      : 1.79769e+308
DBL_MIN      : 2.22507e-308
-DBL_MAX     : -1.79769e+308
Precision value: 6
```



Variable Declaration

- Tells the compiler: “This variable **exists** somewhere.”
- No memory is allocated.

Variable Definition

- Actually **creates** the variable (allocates memory).
- Must appear **once only** in the program.

Rule of Thumb:

Declaration = *Promise*

Definition = *Creation*



The **extern** Keyword

- Tells the compiler: “This variable is **defined somewhere else.**”
- Used to **share variables across files**

Local Variable

- Declared **inside a function or block**
- **Exists only** while the function is executing
- Stored on the **stack**

Global Variable

- Declared **outside** all functions
- Accessible from **any file** (if properly declared)
- Stored in **data/BSS memory sections**

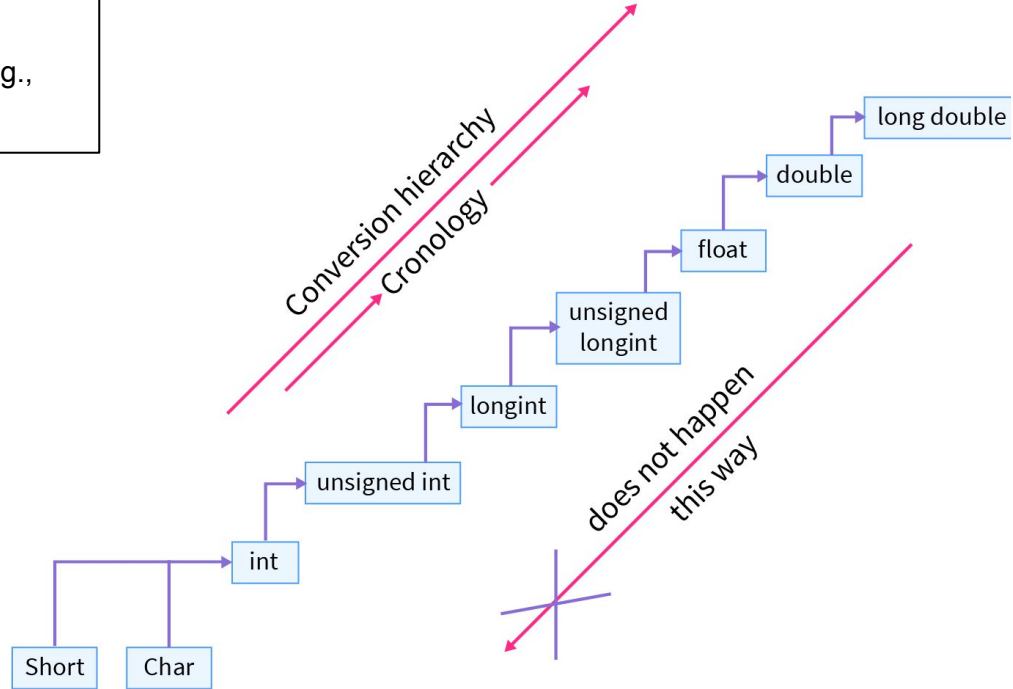


1. Implicit Type Casting (Type Promotion)

- Happens **automatically** by the compiler
- Converts smaller types to larger types (e.g., char to int, float to double)

2. Explicit Type Casting

- **You manually** tell the compiler to convert the type
- Syntax: (new_type) value



! Type Conversion Mismatch in C

When converting data from one type to another — either **implicitly** or **explicitly** — you may encounter **unexpected results** or even **system crashes**, especially in embedded applications.

Let's look at the 3 main risks:

1. Truncation

Truncation occurs when **converting a larger type to a smaller type**, and the extra bits **don't fit** — so they're simply discarded.

```
#include <stdio.h>

int main() {
    int x = 300;
    char y = (char)x;

    printf("x = %i\n", x); // x=300
    printf("y = %i\n", y); // y=44

    return 0;
}
```

Data Type	Binary (Decimal 300)
int	00000001 00101100 (300)
char	00101100 = 44



2. Sign Mismatch

This happens when converting between **signed** and **unsigned** types, which can lead to **unexpected negative or huge values** due to bit interpretation.

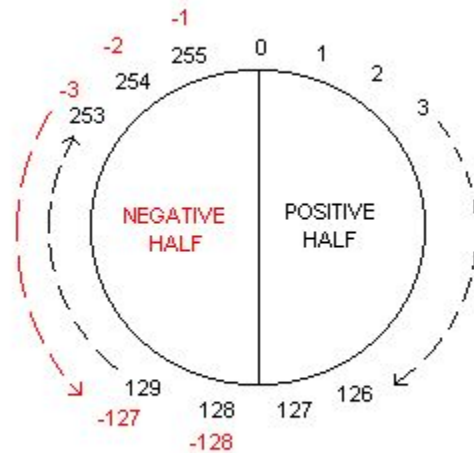
```
#include <stdio.h>
#include <stdint.h>
int main() {

    uint8_t a = 250;
    int8_t b = (int8_t)a; // b = -6!

    printf("a = %i\n", a); //a=250
    printf("b = %i\n", b); //b=-6

    return 0;
}
```

UNSIGNED NUMBERS: 0 to 255
SIGNED NUMBERS: -1 to -128 and 0 to +127



8-BIT NUMBER SYSTEM



- **a** is **unsigned 8-bit**, so it can represent: **0 to 255**
- **b** is **signed 8-bit**, so it can represent: **-128 to 127**

The important point is that the **bits don't change** during the conversion.

- **Step 1: 250 in binary** : $250 = 1\ 1\ 1\ 1\ 1\ 0\ 1\ 0$
- **Step 2: Interpret those same bits as `int8_t`**

For a signed 8-bit integer using two's complement: $1\ 1\ 1\ 1\ 1\ 0\ 1\ 0$

- The first bit is 1, so the number is negative.
- To find its value: $1\ 1\ 1\ 1\ 1\ 0\ 1\ 0$
 - invert $\rightarrow 00000101$
 - add 1 $\rightarrow 00000110$

00000110 = 6, so the result is: -6

3. Overflow

When the result of an operation **exceeds the maximum value** that a variable type can hold.

```
#include <stdio.h>
#include <stdint.h>
int main() {

    uint8_t a = 200;
    uint8_t b = 100;
    uint8_t result = a + b; // result = 44 due to overflow (300 % 256)

    printf("a = %i\n", a);
    printf("b = %i\n", b);
    printf("result = %i\n", result);

    return 0;
}
```

```
#include <stdio.h>
#include <stdint.h>
int main() {

    uint8_t a = 200;
    uint8_t b = 100;
    uint8_t result = a + b; // result = 44 due to overflow (300 % 256)

    printf("a = %i\n", a);
    printf("b = %i\n", b);
    printf("result = %i\n", result);

    uint16_t correctResult = (uint16_t)a + (uint16_t)b;
    printf("Correct Result = %i\n", correctResult);

    return 0;
}
```



Identifiers Modifications

1. Type Modifiers

These **modify the size or sign** of data types.

Modifier	Applies to	Effect
short	int	Reduces size (typically 16 bits)
long	int, double	Increases size (usually 32 or 64 bits)
signed	int, char	Allows negative values
unsigned	int, char	Only positive values (no sign bit)



2. Type Qualifiers

These define **how variables are accessed or modified**, especially across time and multiple files.

◆ **const**

- Variable **cannot be modified** after initialization
- Stored in **Flash/ROM** in embedded systems if global and in stack in local scope

◆ **volatile**

- Tells the compiler: "**This variable can change unexpectedly**"
- Prevents compiler **optimization**

Used when:

- A variable is **modified by an interrupt**
- Reading from **memory-mapped registers** (e.g., GPIO, timers)
- Interfacing with **peripherals**



3. Storage Class / Memory Qualifiers

These control **where** the variable is stored, **lifetime**, and **visibility**.

Qualifier	Scope	Lifetime	Location	Used For
auto	Local	Auto	Stack	Default for local vars
static	Local/Global	Permanent	Data segment	Keeps value between function calls
register	Local	Auto	CPU Register*	Suggests fast access (rare today)
extern	Global	Permanent	Shared across files	Access global variable defined elsewhere

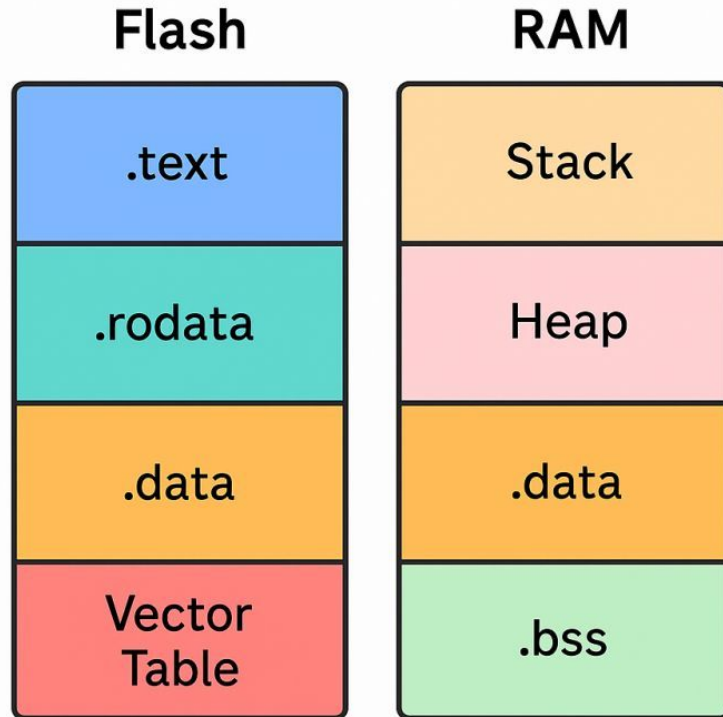


Literals in C

Literal Type	Format / Prefix	Suffix	Description	Example
Integer (decimal)	None	u / U	Unsigned integer	100u
		l / L	Long integer	1000L
		ul / UL	Unsigned long integer	1000UL
Integer (octal)	Leading 0	Same as decimal	Octal (base 8)	075
Integer (hex)	Leading 0x or 0X	Same as decimal	Hexadecimal (base 16)	0xFFU
Floating-point	Number with decimal point	f / F	float (single precision)	3.14f
	Number with decimal point	l / L	long double	2.0L
	Exponent notation (e.g., e3)	Same as above	Scientific notation	1.5e3f
Character	Single quotes ' '	None	Single character	'A'
String	Double quotes " "	None	String of characters	"Hello"



Memory Layout in Embedded C



The main reason we use **memory segmentation** is that different types of program data have **different purposes, lifetimes, permissions, and storage requirements**.



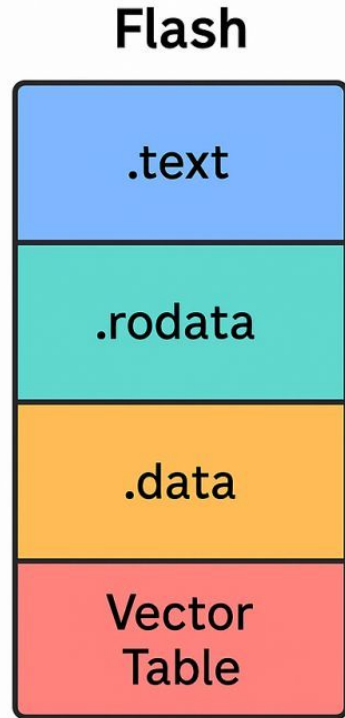
Memory Layout in Embedded C

Why Memory is Divided into Segments/Sections

- **Different lifetimes** – Some data lives for the whole program (globals), some only during a function call (stack), some until explicitly freed (heap), and some must survive power-off (Flash).
- **Different writability** – Code and constants should never change at runtime, so they go in read-only memory (Flash/ROM). Variables that change need writable RAM.
- **Different initialization needs** – Variables with a starting value (`int x = 5;`) need that value stored in Flash and copied to RAM at boot. Uninitialized variables just need zeroed space — no stored value required.
- **Different memory types/speeds** – Embedded systems often have multiple physical memories (internal SRAM, external SDRAM, Flash, TCM). Segmentation lets critical code/data sit in fast memory while bulk data goes to slower/larger memory.
- **Tight resource constraints** – Embedded RAM/Flash is limited (KBs). Clear segmentation lets the linker calculate usage per section and catch overflows at build time.



Memory Layout in C and Embedded C



Section / Region	What is stored	Example
<code>.text</code>	Program instructions / machine code	Compiled functions
<code>.rodata</code>	Read-only constants and data	String literals, lookup tables, const data
<code>.data</code> initializers	Initial values for writable global/static variables	<code>int x = 10;</code> initial value 10
Vector Table	Reset and interrupt handler addresses	Reset, Timer IRQ, UART IRQ
Bootloader <i>(if present)</i>	Startup and firmware-update code	Bootloader program
Configuration / Calibration Data <i>(if used)</i>	Persistent device-specific values	Calibration coefficients, device settings

Lookup table (LUT): A predefined array of values stored in memory so the program can quickly retrieve a result instead of calculating it at runtime.



RAM



Section / Region	What Is Stored?	Examples	Why Is It Stored Here?
.data	Initialized global and static variables that can be modified	<code>int x = 10;</code> <code>static int speed = 50;</code>	These variables need to be writable during program execution
.bss	Uninitialized or zero-initialized global and static variables	<code>int counter;</code> <code>static int buffer[100];</code>	They need RAM during execution; startup code initializes them to zero
Stack	Function-call data, local variables, parameters, return information, saved registers	<code>int x = 10;</code> inside a function	Provides automatic memory for function calls and is typically allocated/released as functions execute
Heap (if used)	Dynamically allocated memory	<code>malloc()</code> , <code>calloc()</code> , <code>realloc()</code>	Allows memory to be allocated and released at runtime
DMA Buffers (if used)	Data buffers used by DMA peripherals	UART, SPI, ADC, Ethernet buffers	Peripherals can read/write these buffers directly without the CPU copying every byte
Peripheral / Shared Buffers (if used)	Data shared between software and hardware or between tasks	Communication buffers, sensor data	Allows different parts of the system to exchange data while the program is running



```
#include <stdio.h>

void test_locals() {

    // Plain local variable
    int local_var = 1;

    // const local variable
    const int const_local_var = 2;

    // static local variable
    static int static_local_var = 0;

    // Modify them
    local_var++;

    // const_local_var++; // Compile error if uncommented
    static_local_var++;
    printf("\n--- Inside test_locals() ---\n");
    printf("local_var: %d\n", local_var);
    printf("const_local_var: %d\n", const_local_var);
    printf("static_local_var: %d\n", static_local_var);
}

int main() {
    test_locals();
    test_locals(); // Observe how static_local_var persists
    return 0;
}
```



Lab 1 :

- Write a C program to compute the perimeter and area of a rectangle with a height of 7 inches and width of 5 inches.
- Write a C program to convert specified days into years, weeks and days.
- Write a C program to calculate the distance between two points (x_1, y_1) , (x_2, y_2) .
- Write a C program to convert a given integer (in seconds) to hours, minutes and seconds.
- Write a C program that reads two integers and checks if they are multiplied or not.
- Write a C program that reads an integer between 1 and 12 and prints the month of the year in English.
- Write a C program to print all numbers between 1 and 100 which are divided by a specified number.



C - Operators

An **operator** is a symbol that tells the compiler to perform specific mathematical or logical functions. By definition, an **operator** performs a certain operation on operands. An operator needs one or more operands for the operation to be performed.

1. Arithmetic Operators

Operator	Description
+	Adds two operands.
-	Subtracts second operand from the first.
*	Multiplies both operands.
/	Divides numerator by denominator.
%	Modulus Operator and remainder of after an integer division.
++	Increment operator increases the integer value by one.
--	Decrement operator decreases the integer value by one.



```
#include <stdio.h>
```

```
int main(){
```

```
    int op1 = 10;
```

```
    float op2 = 2.5;
```

```
    printf("Operand1: %d Operand2: %f\n", op1, op2);
```

```
    printf("Addition of op1 and op2: %f\n", op1 + op2);
```

```
    printf("Subtraction of op2 from op1: %f\n", op1 - op2);
```

```
    printf("Multiplication of op1 and op2: %f\n", op1 * op2);
```

```
    printf("Division of op1 by op2: %f\n", op1/op2);
```

```
    return 0;
```

```
}
```

```
#include <stdio.h>
```

```
int main(){
```

```
    char op1 = 'F';
```

```
    int op2 = 3;
```

```
    printf("operand1: %c operand2: %d\n", op1, op2);
```

```
    printf("Addition of op1 and op2: %d\n", op1 + op2);
```

```
    printf("Subtraction of op2 from op1: %d\n", op1 - op2);
```

```
    printf("Multiplication of op1 and op2: %d\n", op1 * op2);
```

```
    printf("Division of op1 by op2: %d\n", op1/op2);
```

```
    return 0;
```

```
}
```



2. Relational Operators in C

Operator	Description	Example
==	Checks if the values of two operands are equal or not. If yes, then the condition becomes true.	(A == B)
!=	Checks if the values of two operands are equal or not. If the values are not equal, then the condition becomes true.	(A != B)
>	Checks if the value of left operand is greater than the value of right operand. If yes, then the condition becomes true.	(A > B)
<	Checks if the value of left operand is less than the value of right operand. If yes, then the condition becomes true.	(A < B)
>=	Checks if the value of left operand is greater than or equal to the value of right operand. If yes, then the condition becomes true.	(A >= B)
<=	Checks if the value of left operand is less than or equal to the value of right operand. If yes, then the condition becomes true.	(A <= B)



2. Logical Operators in C

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B)
	Called Logical OR Operator. If any of the two operands is non-zero, then the condition becomes true.	(A B)
!	Called Logical NOT Operator. It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.	!(A)

```
#include <stdio.h>

int main(){

    char a = 'a';
    char b = '\0'; // Null character

    if (a && b){
        printf("Line 1 - Condition is true\n" );
    }

    if (a || b){
        printf("Line 2 - Condition is true\n" );
    }

    return 0;
}
```



2. Bitwise Operators in C

Bitwise operators in C allow low-level manipulation of data stored in computers memory.

Bitwise operators contrast with logical operators in C. For example, the logical AND operator (**&&**) performs AND operation on two Boolean expressions, while the bitwise AND operator (**&**) performs the AND operation on each corresponding bit of the two operands.



Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B)
	Binary OR Operator copies a bit if it exists in either operand.	(A B)
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B)
~	Binary One's Complement Operator is unary and has the effect of 'flipping' bits.	(~A)
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 (x2)
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 (/2)



```
#include <stdio.h>
```

```
int main(){
```

```
    unsigned int a = 60; /* 60 = 0011 1100 */  
    unsigned int b = 13; /* 13 = 0000 1101 */  
    int c = 0;
```

```
    c = a & b;          /* 12 = 0000 1100 */  
    printf("Line 1 - Value of c is %d\n", c );
```

```
    c = a | b;          /* 61 = 0011 1101 */  
    printf("Line 2 - Value of c is %d\n", c );
```

```
    c = a ^ b;          /* 49 = 0011 0001 */  
    printf("Line 3 - Value of c is %d\n", c );
```

```
    c = ~a;             /* -61 = 1100 0011 */  
    printf("Line 4 - Value of c is %d\n", c );
```

```
    c = a << 2;          /* 240 = 1111 0000 */  
    printf("Line 5 - Value of c is %d\n", c );
```

```
    c = a >> 2;          /* 15 = 0000 1111 */  
    printf("Line 6 - Value of c is %d\n", c );
```

```
    return 0;
```

```
}
```



3. Assignment Operators in C

The assignment operator stores a certain value in an already declared variable. A variable in C can be assigned the value in the form of a literal, another variable, or an expression.

The value to be assigned forms the right-hand operand, whereas the variable to be assigned should be the operand to the left of the "=" symbol, which is defined as a simple assignment operator in C.



Operator	Description	Example
=	Simple assignment operator. Assigns values from right side operands to left side operand	$C = A + B$ will assign the value of $A + B$ to C
+=	Add AND assignment operator. It adds the right operand to the left operand and assign the result to the left operand.	$C += A$ is equivalent to $C = C + A$
-=	Subtract AND assignment operator. It subtracts the right operand from the left operand and assigns the result to the left operand.	$C -= A$ is equivalent to $C = C - A$
*=	Multiply AND assignment operator. It multiplies the right operand with the left operand and assigns the result to the left operand.	$C *= A$ is equivalent to $C = C * A$
/=	Divide AND assignment operator. It divides the left operand with the right operand and assigns the result to the left operand.	$C /= A$ is equivalent to $C = C / A$
%=	Modulus AND assignment operator. It takes modulus using two operands and assigns the result to the left operand.	$C \% = A$ is equivalent to $C = C \% A$
<<=	Left shift AND assignment operator.	$C <<= 2$ is same as $C = C << 2$
>>=	Right shift AND assignment operator.	$C >>= 2$ is same as $C = C >> 2$
&=	Bitwise AND assignment operator.	$C \&= 2$ is same as $C = C \& 2$
^=	Bitwise exclusive OR and assignment operator.	$C \wedge= 2$ is same as $C = C \wedge 2$
=	Bitwise inclusive OR and assignment operator.	$C = 2$ is same as $C = C 2$



4. Ternary Operators in C

The ternary operator (?:) in C is a type of conditional operator. The term "ternary" implies that the operator has three operands. The ternary operator is often used to put multiple conditional (if-else) statements in a more compact manner.

exp1 ? exp2 : exp3

- exp1 - A Boolean expression evaluating to `true` or `false`
- exp2 - Returned by the ? `operator` when exp1 is `true`
- exp3 - Returned by the ? `operator` when exp1 is `false`

```
#include <stdio.h>

int main(){
    int a = 10;

    (a % 2 == 0) ? printf("%d is Even \n", a) : printf("%d is Odd \n", a);

    return 0;
}
```



```
#include <stdio.h>
```

```
int main(){
```

```
    int a = 100, b = 20, c;
```

```
    c = (a >= b) ? a : b;
```

```
    printf ("a: %d b: %d c: %d\n", a, b, c);
```

```
    return 0;
```

```
}
```

```
#include <stdio.h>
```

```
int main(){
```

```
    int a = 100, b = 20, c;
```

```
    c = (a >= b) ? printf ("a is larger "), c = a : printf("b is larger "), c = b;
```

```
    printf ("a: %d b: %d c: %d\n", a, b, c);
```

```
    return 0;
```

```
}
```



5. Operator Precedence

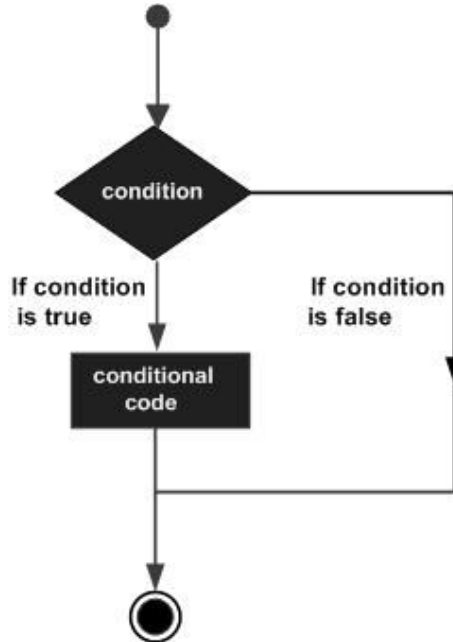
Category	Operator	Associativity
Postfix	() [] -> . ++ --	Left to right
Unary	+ - ! ~ ++ -- (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= >>= <<= &= ^= =	Right to left
Comma	,	Left to right



Decision Making

A computer program is sequential in nature and runs from top to bottom by default. The decision-making statements in C provide an alternative line of execution. You can ask a group of statements to be repeatedly executed till a condition is satisfied.

The decision-making structures control the program flow based on conditions.



Switch Case

The `switch-case` statement is a decision-making statement `in` C. The `if-else` statement provides two alternative actions to be performed, whereas the `switch-case` construct is a multi-way branching statement.

A `switch` statement `in` C simplifies multi-way choices by evaluating a single variable against multiple values, executing specific code based on the match. It allows a variable to be tested `for` equality against a list of values.



Switch Statement Rules in C/C++

1. Expression type:

- The expression in a **switch** must be **integral** (**int**, **char**, etc.) or **enumerated** (**enum**).
- In C++, it can also be a **class type** with a single conversion function to an integral or enumerated type.

2. Case statements:

- You can have **any number of case statements**.
- Each **case** is followed by a **constant value** and a colon (:).

3. Constant expression:

- The value in each **case** must have the **same type** as the switch variable.
- It must be a **constant or literal**.



4. Break statement:

- i. When a **break** is reached, the switch **terminates**.
- ii. Control moves to the statement **after the switch**.

5. Fall-through:

- iii. Not every **case** needs a **break**.
- iv. If **break** is omitted, execution **falls through** to the next case until a break or the switch ends.

6. Default case:

- v. Optional. Executes when **no other case matches**.
- vi. Usually appears at the **end of the switch**.
- vii. No **break** is needed in the default case if it is last.



```
#include <stdio.h>
```

```
int main (){
```

```
    char ch = 'a';
```

```
    printf("Time code: %c\n\n", ch);
```

```
    switch (ch){
```

```
        case 'a':
```

```
            printf("Good Afternoon\n");
```

```
            // break;
```

```
        case 'e':
```

```
            printf("Good Evening\n");
```

```
            // break;
```

```
        case 'm':
```

```
            printf("Good Morning\n");
```

```
    }
```

```
    return 0;
```

```
}
```

```
#include <stdio.h>
```

```
int main() {
```

```
    char ch = 'a';
```

```
    switch (ch) {
```

```
        case '0':
```

```
        case '1':
```

```
            printf("%c is a digit 0 or 1\n", ch);
```

```
            break;
```

```
        case 'a':
```

```
        case 'b':
```

```
            printf("%c is a lowercase letter a, b\n", ch);
```

```
            break;
```

```
        default:
```

```
            printf("%c is a non-alphanumeric character\n", ch);
```

```
    }
```

```
    return 0;
```

```
}
```

```
#include <stdio.h>
```

```
int main (){
```

```
    char ch = 'g';
```

```
    switch (ch){
```

```
        case 'a' ... 'z':
```

```
            printf("%c is a lowercase alphabet\n", ch);
```

```
            break;
```

```
        case 'A' ... 'Z':
```

```
            printf("%c is an uppercase alphabet\n", ch);
```

```
            break;
```

```
        case 48 ... 57:
```

```
            printf("%c is a digit\n", ch);
```

```
            break;
```

```
        default:
```

```
            printf("%c is a non-alphanumeric character\n", ch);
```

```
    }
```

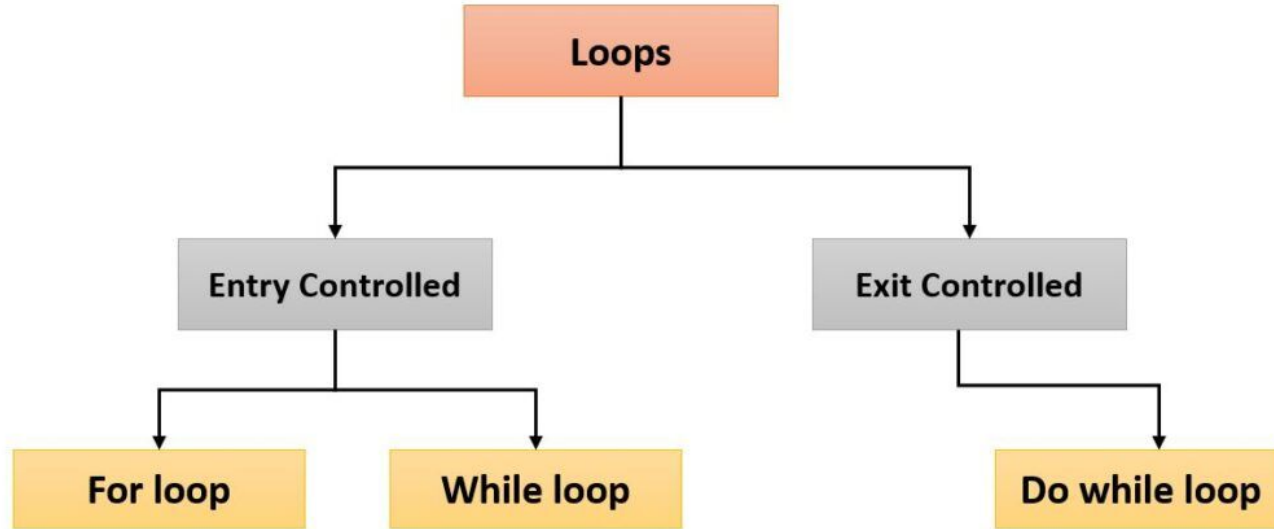
```
    return 0;
```

```
}
```



Loops

Loops are a programming construct that denote a block of one or more statements that are repeatedly executed a specified number of times, or till a certain condition is reached.



For Loops

```
for (init; condition; increment/decrement){  
    statement(s);  
}
```

```
#include <stdio.h>
```

```
int main(){  
    int a = 1;  
  
    for( ; a <= 5; a++){  
        printf("a: %d\n", a);  
    }  
    return 0;  
}
```

```
#include <stdio.h>
```

```
int main(){  
    int a;  
    for(a = 1; a <= 5; ){  
        printf("a: %d\n", a);  
        a++;  
    }  
    return 0;  
}
```

```
#include <stdio.h>
```

```
int main(){  
    int a;  
    for(a = 1; ; a++){  
        printf("a: %d\n", a);  
        if(a == 5)  
            break;  
    }  
    return 0;  
}
```

```
#include <stdio.h>
```

```
int main(){  
    int a, b;  
  
    for(a = 1, b = 1; a <= 5; a++, b++){  
        printf("a: %d b: %d a*b: %d\n", a, b, a*b);  
    }  
    return 0;  
}
```



While Loops

```
while(expression){  
    statement(s);  
}
```

Expression

- An **expression** is something that **produces a value**.
- It can be evaluated to give a **result** (like a number, boolean, or pointer).

Examples:

- `5 + 3` // evaluates to 8
- `x > 0` // evaluates to 1 (true) or 0 (false)
- `a = b + 2` // assignment is also an expression, evaluates to the assigned value

In a `while(expression)` loop, the **expression is evaluated as true (non-zero) or false (0)** to decide whether to enter the loop.



Statement

- A **statement** is an **instruction** that performs an action.
- Statements **do not produce a value** that can be used elsewhere.

Examples:

- `x = x + 1;` `// assignment statement`
- `printf("Hello");` `// function call statement`
- `break;` `// break statement`

- In a **while(expression) { statement(s); }** loop, the statements are **executed repeatedly** as long as the expression is true.

```
while (expr){  
    . . .  
    . . .  
    if (condition)  
    |   break;  
    if (condition)  
    |   continue;  
}
```



Do-While

```
do {  
    statement(s);  
} while(condition);
```

```
#include <stdio.h>  
  
int main(){  
  
    int a = 1;  
  
    do{  
        printf("Hello World\n");  
        a++;  
    } while(a <= 5);  
    printf("End of loop");  
  
    return 0;  
}
```



Nested Loop

```
Outer loop {  
    Inner loop {  
        ...  
        ...  
    }  
    ...  
}
```

```
#include <stdio.h>  
  
int main() {  
    int i, j;  
  
    printf("Program to Print the Tables of 1 to 10 \n");  
  
    // Outer loop: controls the table number (1 to 10)  
    for (i = 1; i <= 10; i++) {  
        // Inner loop: prints each row of the multiplication table for 'i'  
        for (j = 1; j <= 10; j++) {  
            printf("%4d", i * j); /* Print product, width 4 for alignment */  
        }  
  
        printf("\n"); /* Move to the next line after each row */  
    }  
  
    return 0;  
}
```



Infinite Loop

```
#include <stdio.h>

int main() {
    while (1)
    {
        printf("Hello World");
    }

    return 0;
}
```

```
#include <stdio.h>

// infinite for loop
int main(){

    int i;
    for(i=1; ; i++){
        i++;
        printf("Hello World \n");
    }
    return 0;
}
```

```
#include <stdio.h>

// infinite while loop
int main(){

    int i = 0;
    while(i < 10){
        i++;
        printf("Hello World \n");
    }
    return 0;
}
```

```
#include <stdio.h>

int main() {
    do
    {
        printf("Hello World\n");
    } while (1);

    return 0;
}
```



Break Statement

```
while(condition1){  
    . . .  
    . . .  
    if(condition2)  
        break;  
    . . .  
    . . .  
}
```

```
for (init; condition; increment) {  
    . . .  
    if (condition)  
        break;  
    . . .  
}
```

continue Statement

The behaviour of continue statement is opposite to the break statement.

Instead of forcing the termination of a loop, it forces the next iteration of the loop to take place, skipping the rest of the statements in the current iteration.

```
#include <stdio.h>  
  
int main(){  
    int i = 0;  
  
    while (i < 10){  
        i++;  
        if(i%2 == 0)  
            continue; // skip the even  
        printf("i: %d\n", i);  
    }  
}
```



GOTO Statement

1. Purpose:

- The **goto** statement is used to **transfer control** to a defined label within the same function.
- It is an **unconditional jump** (always executes when reached).

2. How it works:

- When **goto** is executed, **program control jumps** to the statement following the label.
- The label must be within the **same function**.

3. Forward jump:

- If the label appears **later in the code**, the **goto** acts as a simple jump forward.

4. Backward jump (looping):

- If the label refers to an **earlier statement**, the **goto** creates a **loop-like behavior**.

5. Important note:

- Overuse of **goto** can make code **hard to read and maintain**.
- It is generally recommended to use structured loops (**for**, **while**) instead.



Why to Avoid goto in C

1. **goto** jumps to any place in the same function, making code **hard to read and follow**.
2. Too many **goto** statements can create **confusing program flow** with multiple entry and exit points.
3. Edsger Dijkstra recommended **avoiding goto** because it can cause **unpredictable behavior**.
4. Many modern languages **don't support goto**, but it still exists in C and C++.
5. **Better alternatives: if-else**, loops (**for**, **while**), **break/continue**, functions, exception handling.
6. **Use goto only** if there's **no other clean solution** for your algorithm.

```
#include <stdio.h>

int main() {
    printf("Before jump\n");
    goto SKIP;

    printf("This will be skipped\n");

SKIP:
    printf("After jump\n");

    return 0;
}
```



Lab 2 :

- Write a C program to print the odd/even numbers for a given range.
- Write a C program that prompts the user to enter a positive integer. It then calculates and prints the factorial of that number
- Write a C program that prompts the user to enter a positive integer. Use a loop to print the multiplication table for that number up to 10.
- Write a program in C to display a pattern like a right angle triangle using an asterisk.
- Write a C program to make such a pattern as a pyramid with an asterisk.
- Write a C program to print the prime numbers for a given range.
- Write a C program to find whether a given year is a leap year or not.
- Write a C program to check whether an alphabet is a vowel or a consonant.
- Write a C program that generates a random number between 1 and 100 and asks the user to guess it.



Functions

- A **function** is a **reusable block of code** that performs a specific task.
- Every C program has at least the **main()** function, but additional functions can be defined for complex tasks.
- Large problems are divided into **smaller, independent blocks** called functions, subroutines, modules, or methods.
- **Function declaration** informs the compiler about the function's name, return type, and parameters.
- **Function definition** contains the **actual code** of the function.
- C provides many **built-in library functions** like `memcpy()`.

Modular Programming

1. Functions enable **modular programming**, where each function handles a specific part of the program.
2. It follows a **top-down approach**: the main routine calls smaller modules (functions).
3. Each function is **independent, complete, and reusable**, returning control (and optionally a result) to the caller.
4. **Advantages**: code becomes **easier to read, develop, maintain, and debug**.



Function Declarations in C

1. A **function declaration** tells the compiler the function's name, return type, and parameters.

```
return_type function_name(parameter list);
```

2. A declaration is needed if a function is defined in one file but called in another.
3. Typically placed at the **top of the file** that calls the function.



Parts of a Function in C

A function definition has:

1. Return Type:

- Data type of value returned by the function (e.g., `int`, `float`).
- If no value is returned → use `void`.

2. Function Name:

- Identifies the function.
- Together with parameter list → forms the **function signature**.

```
return_type function_name(parameter list) {  
    body of the function  
}
```

3. Argument List (Parameters):

- Placeholders for values passed into the function.
- Specifies **type, order, and number** of parameters.
- Parameters are optional.

4. Function Body:

- Block of statements that defines what the function does.



main() Function in C

The **main()** function in C is an entry point of any program. The program execution starts with the **main()** function. It is designed to perform the main processing of the program and clean up any resources that were allocated by the program.

In a C code, there may be any number of functions, but it must have a **main()** function. Irrespective of its place in the code, it is the first function to be executed.

Valid Signatures of main() :

1. `int main()`
2. `int main(void)`
3. `int main(int argc, char *argv[])`

Command-line arguments in C

1. **argc (Argument Count)**
 - An integer that stores the **number of command-line arguments**.
 - Always ≥ 1 because it counts the program name as well.
2. **argv (Argument Vector)**
 - An array of *strings (char)** containing the actual arguments.
 - `argv[0]` → program name (the command used to run the program).
 - `argv[1]` → first argument from user.
 - ...
 - `argv[argc - 1]` → last argument.
 - `argv[argc]` → always **NULL** (marks end of the array).



Important Points about `main()`

1. Every C program **must have a `main()` function** (entry point).
2. `main` is **not a keyword**; it's a user-defined function (body depends on program logic).
3. By convention, `main` returns an **`int`**:
 - `return 0;` → success
 - `return non-zero;` → failure
4. Old compilers allowed `void main()`, but it's **non-standard and not recommended**.
5. Unlike other functions, `main()` cannot:
 - `be inline`
 - `be static`



How `main()` Works

1. Program execution **always starts at `main()`**.
2. `main` can call other functions, passing control (and arguments) to them.
3. Control returns to `main()` when the called function finishes.
4. When `main` finishes, the program ends (unless terminated earlier with functions like `exit()`).



Function Call Mechanisms in C

1. Call by Value (default)

- A **copy** of the actual value is passed to the function.
- Changes inside the function **do not affect** the original variable.

2. Call by Reference (with pointers)

- The **address** of the variable is passed.
- Changes inside the function **do affect** the original variable.

Formal vs Actual Arguments

1. Formal Arguments

```
int add(int x, int y) // x, y → formal arguments
```

- Variables declared **inside the function definition** (parameters).
- Act like local variables within the function.

2. Actual Arguments

- The **real values** or variables passed when calling the function.

```
int result = add(5, 10); // 5, 10 → actual arguments
```



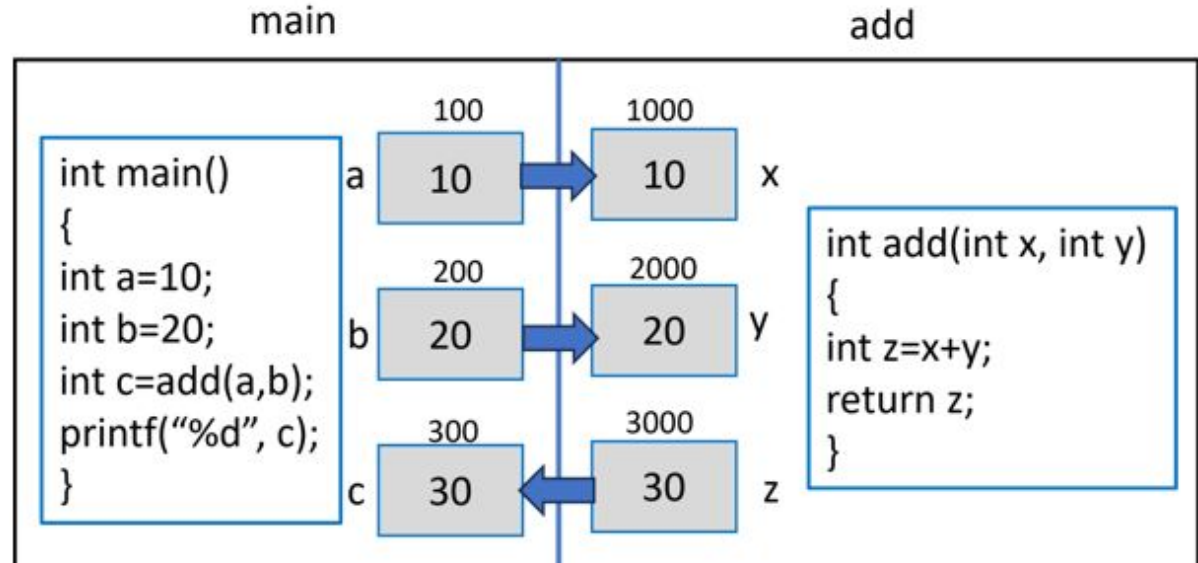
Call by Value

```
#include <stdio.h>

int add(int x, int y){
    int z = x + y;
    return z;
}

int main(){
    int a = 10, b = 20;
    int c = add(a, b);

    printf("Addition: %d", c);
}
```



Local Variables in Function (add)

- **x, y, z** are **local variables** inside **add()**.
- They are created in memory **when the function is called**.
- Each has its own memory address (example: **x** → 1000, **y** → 2000, **z** → 3000).



Local Variables in Function (add)

- x, y, z are **local variables** inside add().
- They are created in memory **when the function is called**.
- Each has its own memory address (example: x → 1000, y → 2000, z → 3000).

Call by Value Process

When add(10, 20) is called:

- Value 10 (from actual argument) is copied into x (formal argument).
- Value 20 (from actual argument) is copied into y.
- So: x (at 1000) = 10 --- y (at 2000) = 20
- Inside add(), the compiler does:
 $z \text{ (at 3000)} = x + y = 10 + 20 = 30$
- Function returns value of z (30) → back to the caller (main).

Back in main()

- Returned value 30 is stored into variable c.
- Local variables x, y, and z are **destroyed** after function ends (they only exist while add() is running).



Call by Address

When a function is called by reference, the address of the actual argument variables passed, instead of their values.

The Address Operator (&) in C

In C language, a variable is a named memory location. When a variable declared, the compiler allocates a random location in the memory and internally identifies the location with the user-defined name.

To fetch the address at which the variable has been created, we use the **address (&) operator**.

Pointer in C

A pointer is a variable that stores the address of another variable. To declare a pointer variable, its name is prefixed with the ***** symbol. The type of the pointer variable and its host variable must be same.

The address is assigned with the **&** operator. The **dereference operator (*)** is used with the pointer. It fetches the value of a variable whose address is assigned to the pointer.



```
#include <stdio.h>

int add(int *, int *);

int main(){
    int a = 10, b = 20;
    int c = add(&a, &b);

    printf("Addition: %d", c);
}

int add(int *x, int *y){
    int z = *x + *y;

    return z;
}
```



1. In **main()**

- Two variables are created: $a = 10$ - $b = 20$

2. Function call happens:

- `add(&a, &b)` is called.
- Here $\&a$ = address of a, $\&b$ = address of b.

3. Inside **add()**

- The addresses of a and b are received by the pointer parameters:
 - x now stores the address of a.
 - y now stores the address of b.

4. Dereferencing with *

- $*x$ means "go to the address stored in x and get its value" $\rightarrow$ value of $a = 10$.
- $*y$ means "go to the address stored in y and get its value" $\rightarrow$ value of $b = 20$.

5. Perform addition : $z = *x + *y \rightarrow z = 10 + 20 = 30$.

6. Return value : Function returns 30 to `main()`.

7. Back in **main()**

- $c = 30$.
- Output: Addition: 30.



Variadic Function in C

A function that can take a variable number of arguments is called a variadic function. One fixed argument is required to define a variadic function.

The most frequently used library functions in C, i.e., `printf()` and `scanf()` are in fact the best-known examples of variadic functions, as we can put a variable number of arguments after the format specifier string.

Syntax of Variadic Functions

In C, a variadic function is defined to have at least one fixed argument, and then followed by an ellipsis symbol (...) that enables the compiler to parse the variable number of arguments.

```
return_type function_name(type fixed_arg, ...);
```

You need to include the `stdarg.h` header file at the top of the code to handle the variable arguments. This header file provides the following macros to work with the arguments received by the ellipsis symbol.



Macro	What it does	In simple words
<code>va_start(ap, lastFixedArg)</code>	Initializes <code>ap</code> so it points to the first variable argument (after the last fixed one).	Start reading extra arguments after the last known argument.
<code>va_arg(ap, type)</code>	Fetches the next argument from the list and converts it to the given type.	Read the next argument (must tell what type it is).
<code>va_copy(dest, src)</code>	Makes <code>dest</code> a copy of <code>src</code> .	Clone the argument list pointer so you can read it separately.
<code>va_end(ap)</code>	Cleans up when you are done with the variable arguments.	Finish and reset the argument list.



```
#include <stdio.h>
#include <stdarg.h>

int sum(int count, ...) {    // `count` is the fixed argument
    va_list ap;
    va_start(ap, count);    // fixed_arg = count

    int total = 0;
    for (int i = 0; i < count; i++) {
        total += va_arg(ap, int);    // read each extra int
    }

    va_end(ap);
    return total;
}

int main() {
    printf("Sum: %d\n", sum(3, 10, 20, 30));
    return 0;
}
```



Callback Function

- A **callback function** is a **function passed as an argument** to another function.
- That other function can **call it (execute it)** whenever needed.
- It's called a “**callback**” because the function is *called back* later, not directly when you pass it.

How does it work

1. **Function pointers** make callbacks possible.
 - A **function pointer** is just a variable that stores the address of a function.
 - With it, we can call the function indirectly.
2. When we **pass a function pointer** to another function, we're letting that function decide *when* and *how* to call our function.



```

#include <stdio.h>

// Step 1: A function we want to use as callback
void myCallback() {
    printf("I am a callback function!\n");
}

// Step 2: A function that takes another function as parameter
void execute(void (*callbackFunc)()) {
    printf("Before callback...\n");
    callbackFunc(); // call the function passed
    printf("After callback...\n");
}

int main() {
    // Step 3: Pass the function name (without parentheses) as argument
    execute(myCallback);
    return 0;
}

```

```

#include <stdio.h>

// Step 1: Define a callback function that takes parameters
void myCallback(int x, int y) {
    printf("Callback executed! Sum = %d\n", x + y);
}

// Step 2: Function that accepts a callback and calls it with arguments
void execute(void (*callbackFunc)(int, int), int a, int b) {
    printf("Before callback...\n");
    callbackFunc(a, b); // Call the callback with given arguments
    printf("After callback...\n");
}

int main() {
    // Step 3: Pass function pointer and parameters
    execute(myCallback, 5, 10);
    return 0;
}

```



Recursive Function in C

A recursive function in C is a function that calls itself. A recursive function is used when a certain problem is defined in terms of itself.

```
1 #include <stdio.h>
2
3 // Recursive function for factorial
4 int factorial(int n) {
5     if (n == 0 || n == 1) // Base case
6         return 1;
7     else
8         return n * factorial(n - 1); // Recursive case
9 }
10
11 int main() {
12     int num = 5;
13     printf("Factorial of %d = %d\n", num, factorial(num));
14     return 0;
15 }
```



Arrays

An **array in C** is a **collection of variables** of the **same data type**, stored in **contiguous (continuous) memory locations**. Think of it like a row of boxes, each box holding one value, all boxes having the same size.

1. Same Data Type

- All elements in an array must have the same type (e.g., all `int`, all `char`, all `float`).

2. Fixed Size

- When you declare an array, you must specify its size (number of elements).
- Example: `int arr[5];` → creates space for **5 integers**.
- You cannot change the size later.

3. Indexing Starts at 0

- First element is `arr[0]`, last element is `arr[size-1]`.
- Example: `arr[0]` is the 1st element, `arr[4]` is the 5th element (if `size = 5`).

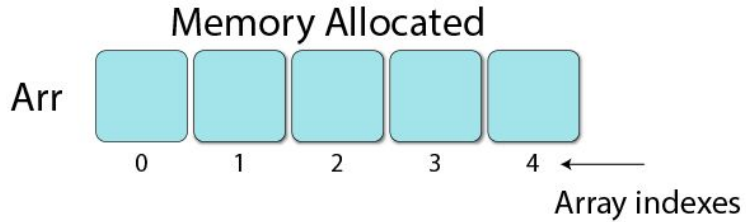
4. Contiguous Memory

- The elements are stored next to each other in memory.
- This makes accessing elements very fast.



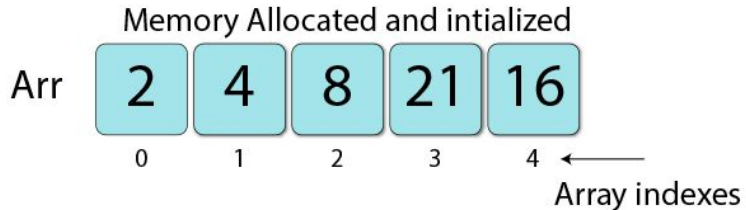
Array Declaration

`Arr [5];` Size of Array no = 5



Array initialization

`Arr [5] = { 2, 4, 8, 12, 16 };`



Multidimensional Arrays

A **multi-dimensional array** in C is simply an **array of arrays**. Useful for **matrices, tables, or grids**

```
type name[size1][size2]...[sizeN];
```

```
int matrix[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

```
int cube[2][2][3] = {  
    {  
        {1, 2, 3},  
        {4, 5, 6}  
    },  
    {  
        {7, 8, 9},  
        {10, 11, 12}  
    }  
};
```

Layer 0 (cube[0]):

```
+-----+  
| [1]  [2]  [3] |  
| [4]  [5]  [6] |  
+-----+
```

Layer 1 (cube[1]):

```
+-----+  
| [ 7]  [ 8]  [ 9] |  
| [10]  [11] [12] |  
+-----+
```

Memory Layout:

- 2 pages
- Each page has 2 rows
- Each row has 3 columns



```
int arr[3][4] = {  
    {1, 2, 3, 4},  
    {5, 6, 7, 8},  
    {9, 10, 11, 12}  
};
```

	Row 0				Row 1				Row 2			
Value	1	2	3	4	5	6	7	8	9	10	11	12
Address	1000	1004	1008	1012	1016	1020	1024	1028	1032	1036	1040	1044



```
int arr[3][5] = {1,2,3,4,5, 10,20,30,40,50, 5,10,15,20,25};
```

Row 0 → [1 2 3 4 5]

Row 1 → [10 20 30 40 50]

Row 2 → [5 10 15 20 25]

arr[0][0] = 1

arr[0][4] = 5

arr[1][2] = 30

arr[2][4] = 25

```
int arr[][5] = {1,2,3,4,5, 10,20,30,40,50, 5,10,15,20,25};
```

```
}
```

```
int arr[][5] = {
```

```
{1, 2, 3, 4, 5},
```

```
{10, 20, 30, 40, 50},
```

```
{5, 10, 15, 20, 25}
```

```
};
```



Pass array to function

- Arrays **decay** (automatically convert) into a pointer to their first element when passed to a function.
- This means what actually gets passed is the **address of the first element**, not a full copy.

```
#include <stdio.h>
```

```
int main(){
    int arr[] = {10, 34, 21, 78, 5};
    float avg = average(arr);
    printf("average: %f", avg);
}

float average(int arr[5]){
    int sum=0;
    int i;
    for (i=0; i<5; i++){
        printf("arr[%d]: %d\n", i, arr[i]);
        sum+=arr[i];
    }
    return (float)sum/5;
}
```

```
#include <stdio.h>
```

```
int main(){
    int arr[] = {10, 34, 21, 78, 5};
    int length = sizeof(arr)/sizeof(int);
    float avg = average(arr, length);
    printf("average: %f", avg);
}

float average(int arr[], int length){
    int sum=0;
    int i;
    for (i=0; i<length; i++){
        printf("arr[%d]: %d\n", i, arr[i]);
        sum+=arr[i];
    }
    return (float)sum/length;
}
```



```
#include <stdio.h>
int main() {
    int a[] = {1,2,3,4,5};
    int *x = a;
    int i;
    for (i=0; i<5; i++){
        printf("%d\n", *x);
        x++;
    }
    return 0;
}
```

Explanation:

1. `int a[] = {1,2,3,4,5};` Declares an array `a` of size 5.
2. `int *x = a;`
 - Declares a pointer `x` that points to the first element of `a`.
 - `a` decays to the address of its first element (`&a[0]`).

So initially:

`x → a[0]` (value = 1 at addr 1000)

3. `int i;` Declares loop counter variable.
4. **Inside loop:**

- `printf("%d\n", *x);`

- `*x` means **value at the address stored in `x`** (dereferencing the pointer).
- On first iteration: `*x = 1`, then `*x = 2`, and so on.

- `x++;`

- Moves the pointer `x` to the next integer in memory.
- Since `int` usually takes 4 bytes, `x` increases by 4 bytes each time.
- So `x` walks through the array.

Memory layout (assume addresses for clarity):

```
a[0] = 1 (addr 0x1000)
a[1] = 2 (addr 0x1004)
a[2] = 3 (addr 0x1008)
a[3] = 4 (addr 0x1012)
a[4] = 5 (addr 0x1016)
```



Return array from a function

```
#include <stdio.h>

int* getArray() {
    static int arr[5] = {10, 20, 30, 40, 50};
    return arr; // return address of static array
}

int main() {
    int *ptr = getArray();
    for (int i = 0; i < 5; i++) {
        printf("%d ", ptr[i]);
    }
    return 0;
}
```



Strings

A **string in C** is a one-dimensional array of char type, with the last character in the array being a "null character" represented by '\0'.

```
char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};  
char greeting[] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

```
#include <stdio.h>  
  
int main (){  
  
    char greeting[20];  
  
    printf("Enter a string:\n");  
    scanf("%s", greeting);  
  
    printf("You entered: \n");  
    printf("%s", greeting);  
  
    return 0;  
}
```

scanf("%s") reads characters until it encounters a whitespace (space, tab, newline, etc.) or EOF.

So, if you try to input a string with multiple words (separated by whitespaces), then the C program would accept characters before the first whitespace as the input to the string.

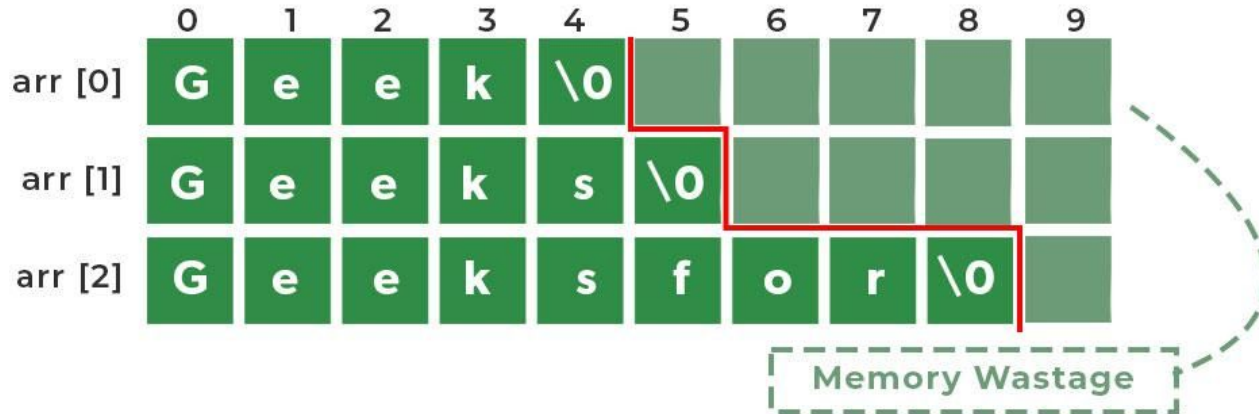


Array of Strings

```
char programming [10][15] = {  
    "PYTHON", "JAVASCRIPT", "PHP",  
    "NODE JS", "HTML", "KOTLIN", "C++",  
    "REACT JS", "RUST", "C"  
};
```

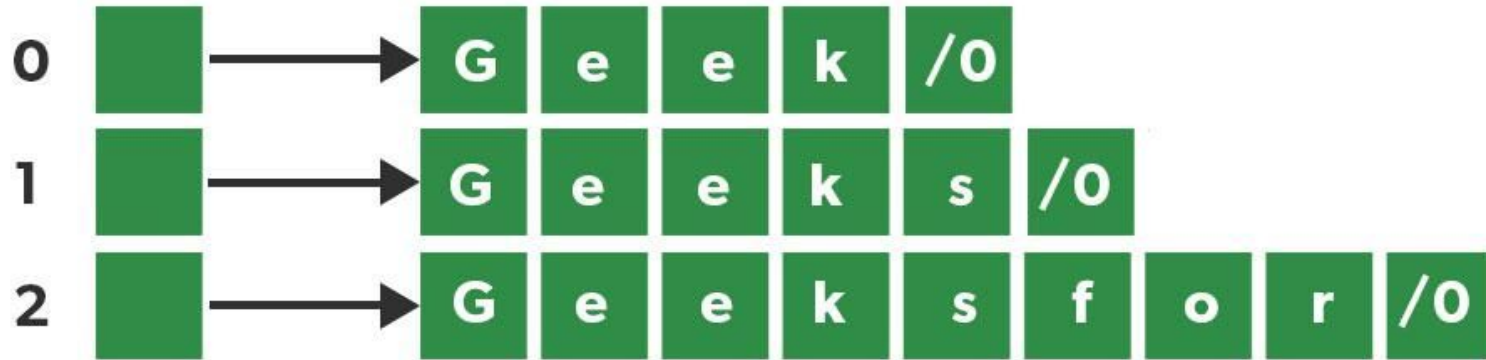
```
char arr[3][10] = {"Geek", "Geeks", "Geekfor"};
```

Memory Representation of an Array of Strings



```
// Array of pointers to strings  
char *arr[] = {"Geek", "Geeks", "Geekfor"};
```

Array of Pointers



No Memory Wastage



Most Used String Functions in C

1. Copying Strings

- `strcpy(dest, src)` → copies one string into another (including `\0`).

```
char s1[20], s2[] = "Hello";  
strcpy(s1, s2); // s1 = "Hello"
```

- `strncpy(dest, src, n)` → copies at most `n` characters. (Safer to avoid overflow, but does not always null-terminate). If the source string length is greater or equal to `n`, the copied result won't end with `\0` automatically.

```
#include <stdio.h>  
#include <string.h>
```

```
int main() {  
    char src[] = "HelloWorld";  
    char dest[6]; // small buffer  
  
    strncpy(dest, src, 5); // copy only 5 chars  
    printf("%s\n", dest); // <-- undefined behavior!  
    return 0;  
}
```

- It copies "Hello" into `dest`
- BUT no `'\0'` is added because we limited it to 5 chars.
- Now `dest` is **not null-terminated** → printing it with `%s` makes `printf` keep reading memory past "Hello" until it finds some random `'\0'`.



2. Concatenating Strings

- `strcat(dest, src)` → appends `src` at the end of `dest`.
- `strncat(dest, src, n)` → appends at most `n` chars.

```
char s1[20] = "Hello ";  
char s2[] = "World";  
strcat(s1, s2);    // s1 = "Hello World"
```

3. Comparing Strings

- `strcmp(s1, s2)` → compares two strings (0 if equal, <0 if `s1`<`s2`, >0 otherwise).
- `strncmp(s1, s2, n)` → compares first `n` chars.

```
if(strcmp("abc", "abc")==0) printf("Equal");
```

4. Finding String Length

- `strlen(s)` → returns number of characters (excluding `\0`).

```
printf("%lu", strlen("Hello"));    // Output: 5
```



5. Searching in Strings

- `strchr(s, c)` → finds first occurrence of character `c`.
- `strrchr(s, c)` → finds last occurrence of character `c`.
- `strstr(s1, s2)` → finds first occurrence of substring `s2` inside `s1`.

6. Tokenizing Strings

- `strtok(s, delim)` → splits a string into tokens using delimiters.

```
char str[] = "red,green,blue";  
char *token = strtok(str, ",");  
while(token != NULL) {  
    printf("%s\n", token);  
    token = strtok(NULL, ",");  
}
```



7. Memory Operations

- `memcpy(dest, src, n)` → copies `n` bytes from `src` to `dest`.
- `memmove(dest, src, n)` → safer copy (handles overlapping memory).
- `memcmp(s1, s2, n)` → compares first `n` bytes.
- `memset(s, c, n)` → fills memory with character `c`.

```
#include <stdio.h>
#include <string.h>

int main() {
    char src[] = "Hello";
    char dest[10];

    memcpy(dest, src, 6); // copy 6 bytes (including '\0')
    printf("dest: %s\n", dest); // Output: Hello
    return 0;
}
```



```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char str[20] = "HelloWorld";

    // Overlapping copy: move "World" over "Hello"
    memmove(str, str+5, 6); // 6 includes '\0'
    printf("str: %s\n", str); // Output: World
    return 0;
}
```

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char a[] = "Hello";
    char b[] = "Hella";

    if (memcmp(a, b, 5) == 0)
        printf("Equal\n");
    else
        printf("Not Equal\n"); // Output: Not Equal
    return 0;
}
```

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char buffer[10];

    memset(buffer, '*', 9); // fill first 9 bytes with '*'
    buffer[9] = '\0';      // null terminate
    printf("buffer: %s\n", buffer); // Output: *****
    return 0;
}
```



Lab 3

- Write a program in C to read n number of values in an array and display them in reverse order.
- Write a program in C to copy the elements of one array into another array.
- Write a program in C to count the total number of duplicate elements in an array.
- Write a program in C to print all unique elements in an array.
- Write a program in C to count the frequency of each element of an array.
- Write a program in C to find the maximum and minimum elements in an array.
- Write a program in C to sort elements of an array in ascending order.
- Write a program in C to sort the elements of the array in descending order.
- Write a program in C to convert a decimal number to a binary number.
- Write a program in C to count the total number of words in a string
- Write a program in C to find the length of a string without using library functions.
- Write a program in C to compare two strings without using string library functions.
- Write a program in C to copy one string to another string.
- Write a program in C to read a sentence and replace lowercase characters with uppercase and vice versa.



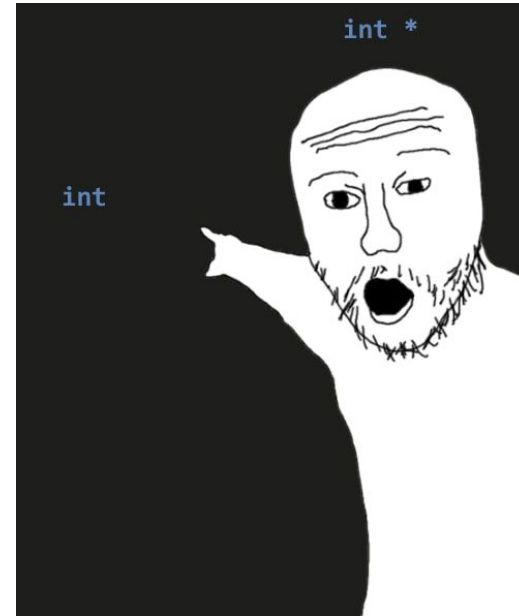
Pointers

A **pointer** in C is a special type of variable that **stores the memory address** of another variable.

Instead of holding a direct value (like 10, 3.14, or 'A'), a pointer holds the *location in memory* where that value is stored.

```
type *pointer_name;
```

- **type** → the data type of the variable the pointer points to
- ***** → declares it as a pointer
- **pointer_name** → the variable name

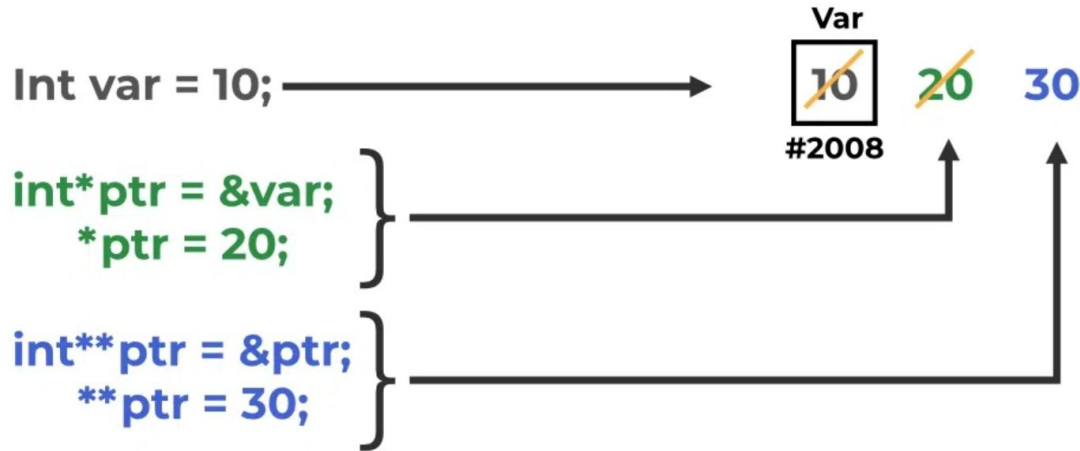


Pointer Size

The **size of a pointer** in C does **NOT** depend on the type of data it points to. It depends only on the **system architecture** (CPU + compiler + OS).

1. On a **32-bit system** → pointers are usually **4 bytes** (can store addresses up to $2^{32} = 4$ GB).
2. On a **64-bit system** → pointers are usually **8 bytes** (can store addresses up to 2^{64} , huge memory space).

How Double Pointer Works in C



```
int main() {  
    int num = 10;           // normal variable  
    int *p = &num;         // pointer storing address of num  
  
    printf("Value of num: %d\n", num);    // 10  
    printf("Address of num: %p\n", &num); // some address  
    printf("Value stored in pointer p: %p\n", p); // same address  
    printf("Value pointed to by p: %d\n", *p); // 10  
  
    return 0;  
}
```

1. `int num = 10;` → num stores the value **10**.
2. `int *p = #` → p is a pointer that stores the **address of num**.
3. `*p` → this is called **dereferencing**, which means accessing the value stored at the address.



Pointer Types

1. Null Pointer

A **null pointer** is a pointer that does not point to any valid memory location. Instead, it holds a special value — usually 0 (or (void *)0) — which means "**points to nothing**".

#define NULL ((void*)0)

```
#include <stdio.h>
int main() {
    int *p = NULL;
    if (p == NULL)
        printf("Null Pointer: does not point to anything.\n");
    return 0;
}
```

Why do we need it?

- To indicate that a pointer is **not assigned** yet.
- To avoid accessing random (garbage) memory.
- Useful for checking before using a pointer.



2. Void Pointer (Generic Pointer)

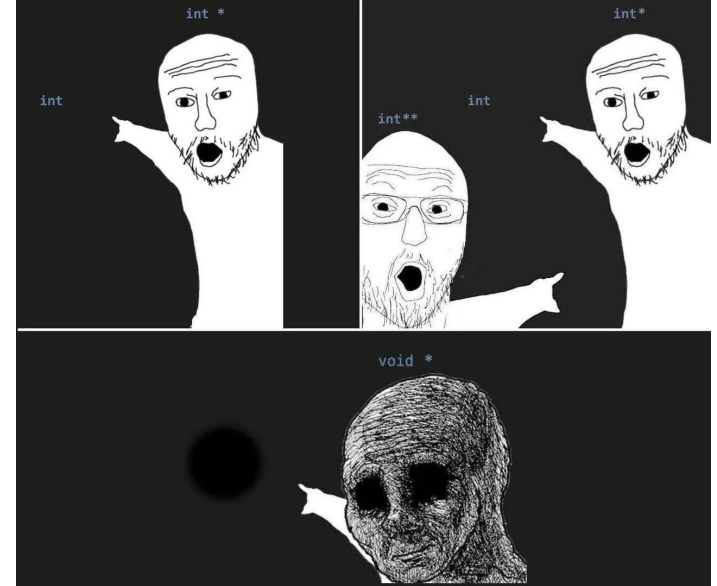
- Special pointer type `void *` that can hold the address of any data type.
- Must be typecast before dereferencing.

```
#include <stdio.h>
int main() {
    int x = 10;
    float y = 5.5;
    void *ptr;

    ptr = &x;
    printf("Value of x = %d\n", *(int *)ptr);

    ptr = &y;
    printf("Value of y = %.1f\n", *(float *)ptr);

    return 0;
}
```



3. Dangling Pointer

Pointer pointing to memory that has been freed or is out of scope.

- De-allocation of memory
- Accessing an out-of-bounds memory location
- When a variable goes out of scope

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p = (int *)malloc(sizeof(int));
    *p = 42;
    free(p);    // memory freed
    // p is now dangling (points to invalid memory)
    p = NULL;   // fix: make it NULL
    return 0;
}
```



4. Wild Pointer

- Uninitialized pointer (points to a random location).

```
#include <stdio.h>
int main() {
    int *p;    // wild pointer (not initialized)
    // *p = 10; // ✗ dangerous, undefined behavior
    p = NULL;  // ✓ good practice
    return 0;
}
```

5. Constant Pointer

- Pointer itself **cannot change** the address it holds.

```
#include <stdio.h>
int main() {
    int a = 10, b = 20;
    int * const p = &a; // constant pointer
    *p = 15; // allowed (value can change)
    // p = &b; // ✗ error (address can't change)
    return 0;
}
```



6. Pointer to Constant

- Pointer can move to other variables, but **cannot modify the value** it points to.

```
#include <stdio.h>
int main() {
    int a = 10, b = 20;
    const int *p = &a;
    // *p = 15; // ✗ error (can't modify value)
    p = &b;      // ✓ allowed (can point elsewhere)
    return 0;
}
```

7. Function Pointer

- Holds the address of a function, used for callbacks.

```
#include <stdio.h>
void hello() {
    printf("Hello, Function Pointer!\n");
}
int main() {
    void (*fptr)() = hello;
    fptr(); // call function using pointer
    return 0;
}
```



Pointer Arithmetic

1. Increment and Decrement of a Pointer

When you increment ($p++$) or decrement ($p--$) a pointer, it moves **forward or backward** by the size of the data type it points to.

```
#include <stdio.h>
int main() {
    int arr[3] = {10, 20, 30};
    int *p = arr;

    printf("Address: %p, Value: %d\n", p, *p);    // first element (10)
    p++;    // move to next element
    printf("Address: %p, Value: %d\n", p, *p);    // second element (20)
    p--;    // move back
    printf("Address: %p, Value: %d\n", p, *p);    // back to first element (10)

    return 0;
}
```

- The pointer moves **+4 bytes** or **-4 bytes** (on 32-bit system) because `int` = 4 bytes.



2. Addition and Subtraction of Integer to Pointer

You can add or subtract an integer `n` from a pointer. This moves the pointer **`n` elements forward or backward**.

```
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = arr;

    printf("Value: %d\n", *(p + 2)); // jumps 2 elements → 30
    printf("Value: %d\n", *(p + 4)); // jumps 4 elements → 50
    printf("Value: %d\n", *(p + 0)); // same as *p → 10

    return 0;
}
```

- Here, `(p + 2)` means **move `2 * sizeof(int)` bytes ahead**.



3. Subtraction of Pointers

You can subtract one pointer from another **if both point to elements of the same array**. The result is the **number of elements between them**.

```
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = &arr[4];    // points to last element (50)
    int *q = &arr[1];    // points to second element (20)

    int diff = p - q;    // difference in terms of elements
    printf("Difference = %d\n", diff);

    return 0;
}
```

- Output: Difference = 3
(Because p is 3 elements ahead of q in the array.)



4. Comparison of Pointers

You can compare pointers with relational operators (`==`, `!=`, `<`, `>`) **only if they point into the same array**. This checks **which element comes first** in memory.

```
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = &arr[1];    // points to 20
    int *q = &arr[3];    // points to 40

    if (p < q)
        printf("p points to an earlier element than q\n");
    else
        printf("p points to a later element than q\n");

    return 0;
}
```



Most common uses of pointers

1. To Access Array Elements

- Array name is itself a pointer to the first element.

```
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = arr; // pointer to array

    for (int i = 0; i < 5; i++) {
        printf("%d ", *(p + i)); // access using pointer arithmetic
    }
    return 0;
}
```



2. For Passing Arguments as Reference

- Allows functions to modify original variables.

```
#include <stdio.h>
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}
int main() {
    int a = 5, b = 10;
    swap(&a, &b);
    printf("a=%d, b=%d\n", a, b); // a=10, b=5
    return 0;
}
```



3. For Passing an Array to Function

```
#include <stdio.h>
void printArray(int *arr, int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
}
int main() {
    int nums[5] = {1, 2, 3, 4, 5};
    printArray(nums, 5);
    return 0;
}
```

4. For Returning Multiple Values from a Function

```
#include <stdio.h>
void calc(int a, int b, int *sum, int *prod) {
    *sum = a + b;
    *prod = a * b;
}
int main() {
    int x = 4, y = 5, s, p;
    calc(x, y, &s, &p);
    printf("Sum=%d, Product=%d\n", s, p);
    return 0;
}
```



5. Allocating Memory Dynamically

- Using `malloc()`, `calloc()`, `realloc()`, and `free()`.

Normally, when you declare variables/arrays in C:

```
int arr[5];    // static allocation
```

- The size of `arr` (5 integers) is fixed at **compile-time**.
- Memory is allocated on the **stack**.
- You **cannot resize** this array later.

But sometimes you don't know in advance how much memory you need (e.g., reading user input, handling files, etc.).

That's where **dynamic memory allocation** comes in.

It allows you to **request memory at runtime** from the **heap** using functions like `malloc`, `calloc`, `realloc`, and `free`.



1. malloc(size)

- Allocates **uninitialized** block of memory of given size (in bytes).
- Returns a pointer to the first byte.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p;
    p = (int*) malloc(5 * sizeof(int)); // allocate space for 5 ints

    if (p == NULL) {
        printf("Memory not allocated\n");
        return 1;
    }

    // assign values
    for (int i = 0; i < 5; i++)
        p[i] = i + 1;

    // print values
    for (int i = 0; i < 5; i++)
        printf("%d ", p[i]);

    free(p); // release memory
    return 0;
}
```

- Memory created in **heap**, not stack.
- Without `free(p)`, you'll cause a **memory leak**.



2. calloc(n, size)

- Allocates memory for **n elements**, each of given size.
- Initializes all bytes to **0**.

```
int *p = (int*) calloc(5, sizeof(int));
```

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)calloc(5, sizeof(int));

    // Checking if failed or pass
    if (ptr == NULL) {
        printf("Allocation Failed");
        exit(0);
    }

    // Print the array
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);
    return 0;
}
```



3. realloc(ptr, new_size)

- Resizes memory previously allocated with malloc or calloc.
- Useful when you need to grow or shrink an array.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)malloc(5 * sizeof(int));

    // Resize the memory block to hold 10 integers
    ptr = (int *)realloc(ptr, 10 * sizeof(int));

    // Check for allocation failure
    if (ptr == NULL) {
        printf("Memory Reallocation Failed");
        exit(0);
    }

    return 0;
}
```



```
int* ptr = (int*) malloc (5* sizeof (int));
```

4 bytes

ptr = 

← 20 bytes of memory →

A large 20 bytes memory block is dynamically allocated to ptr

ptr = realloc (ptr, 10* sizeof(int));

ptr = 

← 40 bytes of memory →

The size of ptr is changed from 20 bytes to 40 bytes dynamically



4. free(ptr)

- Frees (releases) dynamically allocated memory back to the system.
- Always use free after malloc/calloc/realloc to avoid leaks.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)calloc(5, sizeof(int));

    // Do some operations.....
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);

    // Free the memory after completing
    // operations
    free(ptr);

    return 0;
}
```



Array of Pointers

```
#include <stdio.h>

const int MAX = 3;

int main(){

    int var[] = {10, 100, 200};
    int i, *ptr[MAX];

    for(i = 0; i < MAX; i++){
        ptr[i] = &var[i]; // assign the address of integer.
    }

    for (i = 0; i < MAX; i++){
        printf("Value of var[%d] = %d\n", i, *ptr[i]);
    }

    return 0;
}
```



```
#include <stdio.h>

const int MAX = 4;

int main(){

    char *names[] = {
        " Ahmed",
        " Mohamed",
        " Mahmoud",
        " Mostafa"
    };

    int i = 0;

    for(i = 0; i < MAX; i++){
        printf("Value of names[%d] = %s\n", i, names[i]);
    }

    return 0;
}
```

- A pointer to a string **points to the first character** of the string.
- Since strings in C are just arrays of `char` ending with '`\0`', the pointer allows access to the full string from that first character onward.

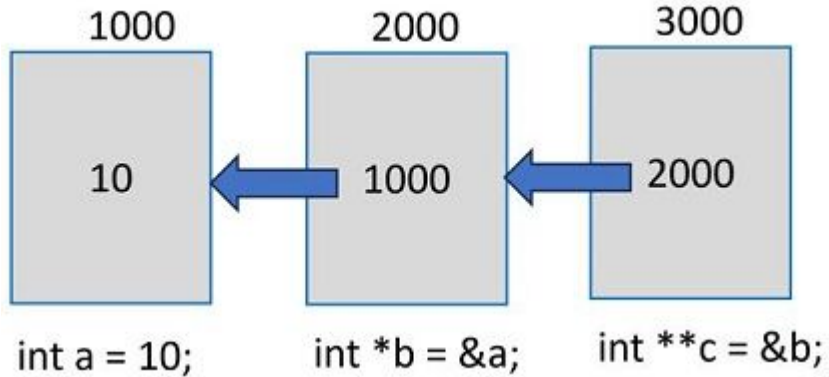
Memory Block: names[] (array of 4 char pointers)

names[0]	' '	'A'	'h'	'm'	'e'	'd'	'\0'		
names[1]	' '	'M'	'o'	'h'	'a'	'm'	'e'	'd'	'\0'
names[2]	' '	'M'	'a'	'h'	'm'	'o'	'u'	'd'	'\0'
names[3]	' '	'M'	'o'	's'	't'	'a'	'f'	'a'	'\0'

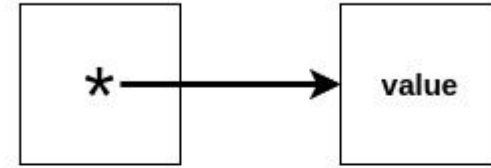


Double Pointer

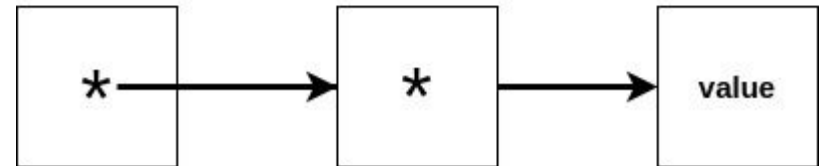
Pointer to pointer (double pointer) used to store the address of another pointer.



Pointer



Double Pointers



```
#include <stdio.h>

int main() {
    // An integer variable
    int a = 100;

    // Pointer to integer
    int *ptr = &a;

    // Pointer to pointer (double pointer)
    int **dptr = &ptr;

    printf("Value of 'a' is : %d\n", a);
    printf("Value of 'a' using pointer (ptr) is : %d\n", *ptr);
    printf("Value of 'a' using double pointer (dptr) is : %d\n", **dptr);

    return 0;
}
```



Function Pointer (pointer to function)

It is a variable that stores the address of a function. Function pointers can be useful when you want to call a function dynamically. The mechanism of callback functions in C is dependent on the function pointers.

Declaration

Syntax **function_return_type** (***Pointer_name**) (**function_argument_list**)

```
#include <stdio.h>

// Defining a function
void hello() { printf("Hello World"); }

// The main code
int main() {
    // Declaring a function pointer
    void (*ptr)() = &hello;

    // Calling function using the
    // function pointer
    (*ptr)();

    return 0;
}
```

```
void hello(){
    printf("Hello World");
}

void (*ptr)() = &hello;
```



```
int add(int a, int b) {  
    return a + b;  
}  
  
int (*func_ptr)(int, int); // Declare the pointer  
func_ptr = add;           // Assign function address  
  
// Call the function using the pointer  
int result = func_ptr(2, 3); // result = 5
```

```
typedef int (*operation_func)(int, int);  
  
int add(int a, int b) { return a + b; }  
  
operation_func f = add;  
int res = f(5, 6); // res = 11
```



Callback Function

A **callback** is a function passed as an **argument** to another function, which then **calls it ("calls back")** during its execution.

It's a common technique in:

- Event handling
- Threading and asynchronous code

simple program that:

- Has a function `process_numbers` that takes a function pointer.
- That function pointer determines **how to process each number**.
- We'll pass two different callbacks: one to **square** numbers, one to **double** them.



```

#include <stdio.h>

// Define callback function type
typedef int (*operation_func)(int);

// Callback function 1: square
int square(int x) {
    return x * x;
}

// Callback function 2: double
int double_val(int x) {
    return 2 * x;
}

// Function that takes a callback and applies it to each array element
void process_numbers(int *arr, int size, operation_func func) {
    for (int i = 0; i < size; i++) {
        arr[i] = func(arr[i]); // call the callback function
    }
}

// Function to print array
void print_array(int *arr, int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

```

```

int main() {
    int numbers[] = {1, 2, 3, 4, 5};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    printf("Original: ");
    print_array(numbers, size);

    // Use square as a callback
    process_numbers(numbers, size, square);
    printf("After square callback: ");
    print_array(numbers, size);

    // Use double_val as a callback
    process_numbers(numbers, size, double_val);
    printf("After double callback: ");
    print_array(numbers, size);

    return 0;
}

```



Array of Function Pointers

An **array of function pointers** lets you store multiple functions (with the same signature) in one place and call them dynamically using an index.

return_type (*ptr[])(args) = {fun1, fun2, ...};

- operations: The name of the array.
- [4]: It holds 4 elements.
- Each element is a pointer to a function that takes two int arguments and returns an int.

```
#include <stdio.h>

// Define function signatures
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }
int divide(int a, int b) { return b != 0 ? a / b : 0; }

// Declare array of function pointers
int (*operations[4])(int, int);

int main() {
    // Assign functions to the array
    operations[0] = add;
    operations[1] = subtract;
    operations[2] = multiply;
    operations[3] = divide;

    int x = 20, y = 5;

    // Call each operation using the function pointer array
    printf("Add: %d\n", operations[0](x, y));    // add(x, y)
    printf("Subtract: %d\n", operations[1](x, y)); // subtract(x, y)
    printf("Multiply: %d\n", operations[2](x, y)); // multiply(x, y)
    printf("Divide: %d\n", operations[3](x, y));  // divide(x, y)

    return 0;
}
```



Passing Pointers to Functions

- Efficiency / Performance (Low Memory Consumption)
- Allows Modification of Original Data
- Allows Sharing of large Data : This helps when **multiple functions** need to access or update the same data.

```
#include <stdio.h>

struct BigData {
    int values[10000]; // Large array
};

// Inefficient: passes whole struct (copies 40,000 bytes)
void processByValue(struct BigData data) {
    data.values[0] = 999;
}

// Efficient: passes just a pointer (4 or 8 bytes)
void processByPointer(struct BigData *data) {
    data->values[0] = 999;
}

int main() {
    struct BigData big = {0};

    processByPointer(&big); // Only address is passed

    printf("First element: %d\n", big.values[0]); // Output: 999
    return 0;
}
```

```
#include <stdio.h>

void updateValue(int *p) {
    *p = 42;
}

int main() {
    int num = 10;
    updateValue(&num);
    printf("Updated num: %d\n", num); // Output: 42
    return 0;
}
```



Return pointer from a function

- Return a Pointer to Dynamically Allocated Memory

```
#include <stdio.h>
#include <stdlib.h>

int* create_array(int size) {
    int *arr = malloc(size * sizeof(int)); // allocate memory on heap
    if (arr != NULL) {
        for (int i = 0; i < size; i++) {
            arr[i] = i * 10;
        }
    }
    return arr; // return pointer to heap memory
}

int main() {
    int *data = create_array(5);

    for (int i = 0; i < 5; i++) {
        printf("%d ", data[i]);
    }
    printf("\n");

    free(data); // don't forget to free heap memory
    return 0;
}
```



- Return a Pointer to a static Local Variable

```
#include <stdio.h>

int* get_static_value() {
    static int x = 100;
    return &x;
}

int main() {
    int *ptr = get_static_value();
    printf("Value: %d\n", *ptr); // Output: 100
    return 0;
}
```

- Return a Pointer Passed into the Function (Modified Input)

```
int* find_max(int *arr, int size) {
    if (size == 0) return NULL;
    int *max = arr;
    for (int i = 1; i < size; i++) {
        if (arr[i] > *max)
            max = &arr[i];
    }
    return max;
}
```



- **Never return a pointer to a local (stack) variable**

```
int* get_local_value() {  
    int x = 42;  
    return &x; // returning address of local variable (stack memory)  
}
```



Returning multiple values from a function using pointers

```
#include <stdio.h>

void calculate_rectangle(int length, int width, int *area, int *perimeter) {
    *area = length * width;
    *perimeter = 2 * (length + width);
}

int main() {
    int length = 10, width = 5;
    int area, perimeter;

    // Call the function and pass addresses of area and perimeter
    calculate_rectangle(length, width, &area, &perimeter);

    printf("Length: %d, Width: %d\n", length, width);
    printf("Area: %d\n", area);
    printf("Perimeter: %d\n", perimeter);

    return 0;
}
```



Character Pointers

Declaring a Character Pointer

```
char *ptr;  
char ch = 'A';  
char *ptr = &ch; // points to the variable ch  
char name[] = "Ahmed";  
char *ptr = name;  
// OR  
char *ptr = &name[0]; // same thing
```

char *ptr = "Hello";

- "Hello" is stored in read-only memory.
- You must not modify the contents via ptr.



Lab 4

- Implement C library string.h



Arrays with pointers

1. 1D Array Access Equation : $\text{arr}[y] = *(\text{arr} + y);$

Explanation:

- `arr` is a pointer to the first element of the array.
- `arr + y` moves the pointer to the y -th element.
- `* (arr + y)` dereferences that element, i.e., gives you `arr[y]`.

2. Calculate Size of 1D Array: $\text{ArraySize} = \text{sizeof}(\text{arr_name}) / \text{sizeof}(\text{arr_name}[0]);$

Conditions:

- Works only if `arr_name` is **declared in the same function** (i.e., not passed as an argument).
- `sizeof(arr_name)` gives total bytes.
- `sizeof(arr_name[0])` gives the size of one element.



3. 2D Array Access (Pointer Form) :

```
arr[index1D][index2D] = *(arr[index1D] + index2D);
```

Why:

- `arr[index1D]` is a pointer to a 1D array (row).
- `arr[index1D] + index2D` moves to the correct column.
- `*` dereferences to access the actual value.

4. Full Pointer Form of 2D Array Access

```
arr[index1D][index2D] = (*(arr + index1D) + index2D);
```

Why:

- `arr` is a pointer to an array of arrays.
- `arr + index1D` gives the address of the `index1D`-th row.
- `*(arr + index1D)` dereferences it to get the row (a pointer to the first column).
- `+ index2D` moves to the correct column.
- Final `*` gets the actual value.



```
int numbers[3][3];           // A 2D array: 3 rows, 3 columns
int (*ptr)[3] = numbers;     // Pointer to an array of 3 integers
```

- `numbers` is a 2D array: type is `int[3][3]`
- When used in expressions, `numbers` decays to a pointer to its first row: `int (*)[3]`
- `ptr` is declared as a pointer to an array of 3 ints: `int (*ptr)[3]`

`ptr` now points to the **first row** of the 2D array:

```
numbers[0] ≡ *(ptr + 0)
numbers[1] ≡ *(ptr + 1)
numbers[2] ≡ *(ptr + 2)
```

- To access an element:

```
numbers[1][2] ≡ (*(ptr + 1) + 2)
```



```

#include <stdio.h>

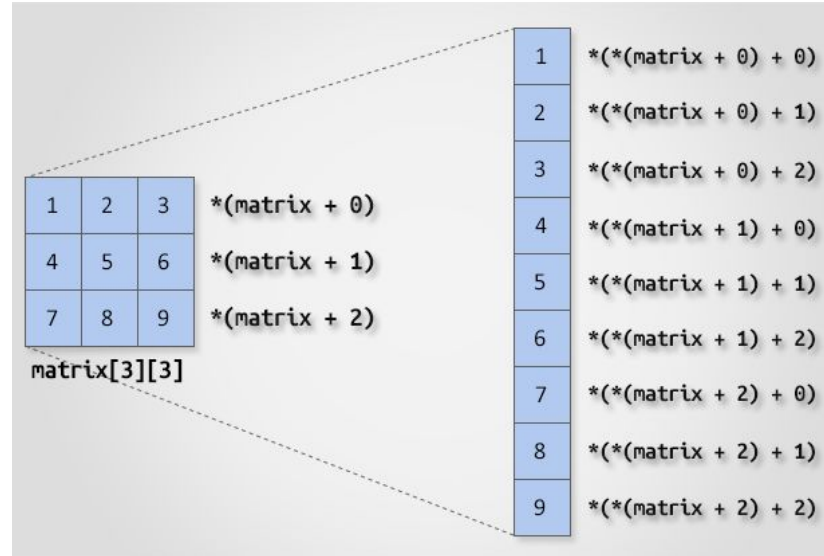
int main() {
    int numbers[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    int (*ptr)[3] = numbers; // Pointer to array of 3 ints

    // Access using ptr
    printf("%d\n", ptr[0][0]); // Output: 1
    printf("%d\n", ptr[1][2]); // Output: 6
    printf("%d\n", *(*ptr + 2) + 1); // Output: 8

    return 0;
}

```



We can know the number of array elements by pointer

```
int arr[ ]={ };
```

- arr: array name
- &arr: array address
- (&arr+1) : address of the next location after the array in memory

We can say array element size : $(* (\& arr + 1) - arr)$



Complex declarations in C

- General Strategy: Clockwise/Spiral Rule
- Start at the variable name, go right first, then left, spiraling outward while following precedence rules.

Operator	Precedence	Associative
(), []	1	Left to Right
*, identifier	2	Right to Left
Data Type	3	



1. `void (*f[10])(int, int);`

- Start at `f`: it's the name.
- `f[10]` → `f` is an array of 10 elements.
- `(*f[10])` → each element is a **pointer**.
- `(*f[10])(int, int)` → each pointer is to a **function taking 2 ints**.
- `void (*f[10])(int, int)` → each function returns `void`.

So, **`f` is an array of 10 pointers to functions that take 2 int arguments and return void.**



2. `char (*(*x())[ ] )();`

- Start at `x`: it's the name.
- `x()` → `x` is a **function** taking no arguments.
- `(*x())` → returns a pointer
- `(*x())[ ]` → to an array (size unspecified)
- `(*x())[ ]()` → each element of the array is a **pointer to a function** that:
- `char (*(*x())[ ] )();` → returns `char`

So, `x` is a function that returns a pointer to an array of pointers to functions returning `char`.



3. `char (*(*x[3])())[5];`

- `x[3]` → `x` is an array of 3 elements
- `(*x[3])()` → each element is a pointer to a **function with no parameters**
- `(*x[3])()` returns a **pointer**
- `(*x[3])()[5]` → to an array of 5 elements
- `char (*(*x[3])())[5]` → each element is of type `char`

So, `x` is an array of 3 pointers to functions that return a pointer to an array of 5 chars.



4. `int *(*arr[5])()`;

- `arr[5]` → `arr` is an array of 5 elements
- `(*arr[5])()` → each is a pointer to a function (no parameters)
- `(**arr[5])()` → the function returns a pointer to a function (no parameters)
- `int *(*arr[5])()` → which returns a pointer to `int`

So, `arr` is an array of 5 pointers to functions that return pointers to functions that return pointer to `int`.



5. `void (*iti(int std, void (*func)(int)))(int);`

- Start at `iti`: it's a **function**
- Takes two parameters:
 - `int std`
 - `void (*func)(int) →` a pointer to a function taking `int`, returning `void`
- `iti(...)` returns:
 - `void (*)(int) →` a pointer to a function taking an `int` and returning `void`

So, `iti` is a function taking an `int` and a pointer to a function (`int → void`), and returning a pointer to a function (`int → void`).



int *ptr;

- ptr is a **pointer to int**

int **ptr;

- ptr is a **pointer to a pointer to int**

int (*ptr)[5];

- ptr is a **pointer to an array of 5 ints**

Operator	Precedence	Associative
(), []	1	Left to Right
*, identifier	2	Right to Left
Data Type	3	



```
int *arr[5];
```

- arr is an **array of 5 pointers to int**

```
int (*func_ptr)(int, float);
```

- func_ptr is a **pointer to a function** taking int and float and returning int

```
int (*arr[3])(char *);
```

- arr is an **array of 3 function pointers**, Each function takes a char * and returns an int

```
char *(*signal(int, void (*)(int)))(int);
```

1. signal is a function, Takes: int and a function pointer void (*)(int)
2. Returns: a pointer to a function: char *f(int)
3. So, signal is a function that takes an int and a function pointer, and returns a **pointer to a function** that takes an int and returns a char *.

Operator	Precedence	Associative
(), []	1	Left to Right
*, identifier	2	Right to Left
Data Type	3	



User Defined Data Type

Structures

A **structure** in C is a user-defined data type that lets you **group variables** of **different types** under a single name. For example, a **book** is described by its **title** (string), **author** (string), **price** (double), **number of pages** (integer), etc. Instead of using four different variables, these values can be stored in a single **struct** variable.

Structure declaration :

```
struct structure_name {  
    dataType1 var_name1;  
    dataType2 var_name2;  
    dataType3 var_name3;  
} [one or more structure variables / opt
```

```
struct book{  
    char title[50];  
    char author[50];  
    double price;  
    int pages;  
} book1; /* book1 is optional*/
```

```
struct {  
    char title[50];  
    char author[50];  
    double price;  
    int pages;  
} book1; /* book1 is a must*/
```



- Structure is allocated in the memory when create variable/object from it **But the declaration only does not allocate a memory location.**
- Struct initialization : `struct book book1 = {"C How to Program", "Deitels'", 100, 200};`

```
#include <stdio.h>

int main() {
    struct book {
        char title[50];
        char author[50];
        int pages;
        int price;
    };

    struct book book1 = {"C How to Program", "Deitels'", 100, 200};

    printf("Title: %s\n", book1.title);
    printf("Author: %s\n", book1.author);
    printf("Pages: %d\n", book1.pages);
    printf("Price: %d\n", book1.price);

    return 0;
}
```



- Structure is allocated in the memory when create variable/object from it **But the declaration only does not allocate a memory location.**
- **A structure cannot contain an object of its own type, either directly or indirectly, except through a pointer.**
- However, a structure **can** contain an object of **another structure** — as long as that structure is **fully defined before** it's used.

1. Not Allowed (Self-Reference via Value):

```
struct Node {  
    int data;  
    struct Node next;    // Error: incomplete type  
};
```

- This creates an **infinite nesting**: each Node would contain another Node, and so on.
- The compiler can't compute the size $\Rightarrow$ **invalid**.



2. Allowed (Self-Reference via Pointer):

```
struct Node {  
    int data;  
    struct Node* next; // Pointer has known, fixed size  
};
```

- Commonly used in linked lists, trees, etc.
- `struct Node*` is just a **pointer**, typically 4 or 8 bytes.



3. Allowed: Structure Inside Another Structure (Not Self-Referencing)

```
struct Address {  
    char city[20];  
    int zip;  
};  
  
struct Person {  
    char name[50];  
    struct Address addr; // Valid: Address is already fully defined  
};
```

- This is **valid** because Address is a **different type** and already fully defined before use.



4. Indirect Self-Reference Is Also Invalid (via value):

```
struct A {  
    struct B b;    // if B contains A by value, this is invalid  
};  
  
struct B {  
    struct A a;    // indirect recursion via value - still infinite  
};
```

Only **pointer references** are allowed in such cases.



- The **size of a struct object** is **greater than or equal to the sum of the sizes of its members**, because of **padding and alignment**.

Structure Member Alignment

Structure member alignment refers to how data members of a structure are arranged in memory to conform to **CPU memory access boundaries** (like 2, 4, 8 bytes).

Why it matters:

Most CPUs **require or prefer** that data types be stored at **addresses that are multiples of their size** (or alignment requirement), to avoid **slower or illegal memory accesses**.

For example:

- `int` (4 bytes) is aligned at an address divisible by 4
- `double` (8 bytes) is aligned at an address divisible by 8

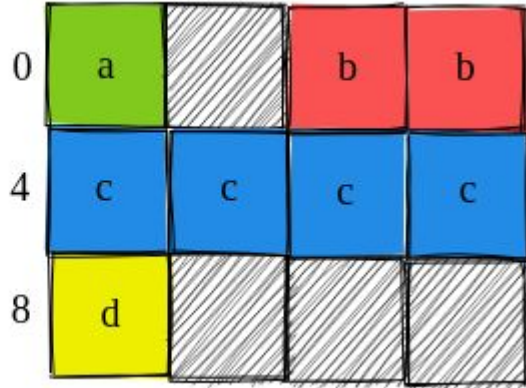


What is Padding

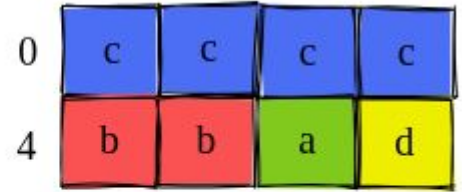
Padding is the **extra unused bytes** the compiler inserts between structure members to **satisfy alignment rules**.

This padding ensures that each member starts at a memory address aligned for its data type.

```
struct foo {  
    char a;  
    uint16_t b;  
    int32_t c;  
    char d;  
};
```



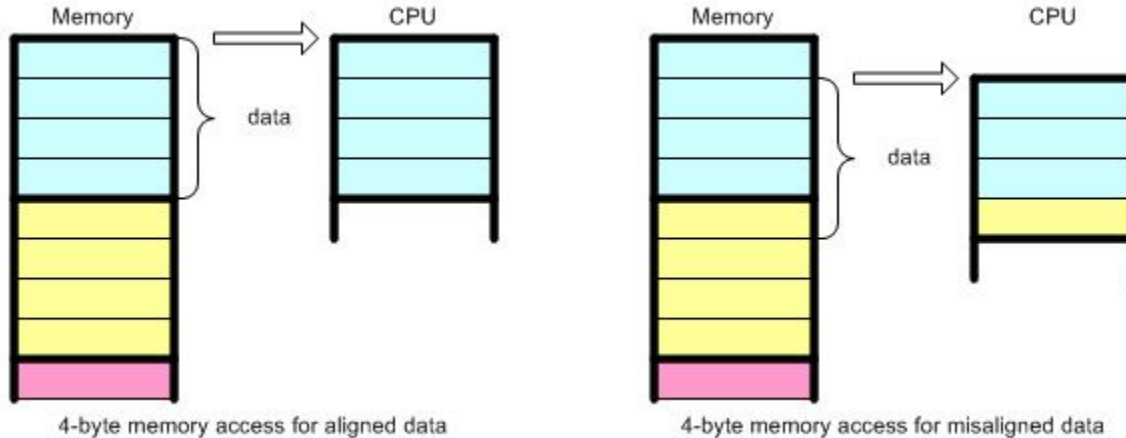
```
struct foo {  
    int32_t c;  
    uint16_t b;  
    char a;  
    char d;  
};
```



Data Packing

Packing is when you **tell the compiler NOT to insert padding** between structure members — packing them tightly together.

This is done to **save space**, especially in embedded systems or protocols — but may lead to **slower access or alignment faults** on some architectures.



The **#pragma pack(1)** preprocessor directive forces the compiler to disregard the padding, and use packing



typedef

- typedef is used to **create a new name (alias)** for an existing data type.
- With struct, it allows you to **avoid repeatedly writing struct** when declaring variables.

```
typedef struct {  
  
    // member declarations  
  
} AliasName;
```

```
struct StructName {  
  
    // members  
};  
  
typedef struct StructName AliasName;
```

```
struct Book {  
    char title[50];  
    char author[50];  
    int pages;  
    float price;  
};  
  
typedef struct Book Book_t;  // Book_t is alias
```



Structures and Pointers

```
struct Person {  
    char name[50];  
    int age;  
};  
  
struct Person p1 = {"Alice", 30};  
struct Person *ptr = &p1;  
  
printf("Name: %s\n", (*ptr).name);    // classic dereference  
printf("Age: %d\n", ptr->age);        // arrow operator (preferred)
```

- Arrow Operator (->) : used with structure pointers to access struct members
- Dot Operator (.) : used with struct objects



Dynamically Allocate Structure

Use `malloc()` when you need to create a structure **at runtime**

```
#include <stdlib.h>

struct Person *p = malloc(sizeof(struct Person));
if (p != NULL) {
    strcpy(p->name, "Bob");
    p->age = 45;
}
```

- Don't forget to `free(p)`; when done.



Structures and Pointers

```
struct Person group[3] = {
    {"Tom", 25},
    {"Jerry", 28},
    {"Spike", 35}
};

struct Person *ptr = group; // points to first element

for (int i = 0; i < 3; i++) {
    printf("%s - %d\n", (ptr + i)->name, (ptr + i)->age);
}
```

```
void printPerson(struct Person *p) {
    printf("Name: %s, Age: %d\n", p->name, p->age);
}

int main() {
    struct Person p = {"Sarah", 40};
    printPerson(&p);
}
```



Structures and Arrays

```
#include <stdio.h>

typedef struct {
    char title[10];
    double price;
    int pages;
}book_t;

int main(){

    book_t b[3];

    strcpy(b[0].title, "Learn C");
    b[0].price = 650.50;
    b[0].pages = 325;

    strcpy(b[1].title, "C Pointers");
    b[1].price = 175;
    b[1].pages = 225;

    strcpy(b[2].title, "C Pearls");
    b[2].price = 250;
    b[2].pages = 325;

    printf("\nList of Books:\n");
    for (int i = 0; i < 3; i++){
        printf("Title: %s \tPrice: %7.2lf \tPages: %d\n", b[i].title, b[i].price, b[i].pages);
    }

    return 0;
}
```



Self-referential Structures

A **self-referential structure** in C is a structure that **contains a pointer to a variable of its own type**.

This is a key building block for many **data structures** like:

- Linked lists
- Trees
- Stacks
- Queues

```
struct typename{  
    type var1;  
    type var2;  
    ...  
    ...  
    struct typename *var3;  
}
```

Because C structures **cannot contain an instance** of themselves (it would cause infinite memory), but **they can contain a pointer** to an instance of the same type.

```
struct Node {  
    int data;  
    struct Node *next; // Pointer to another Node (same type)  
};
```

```
struct Node {  
    int data;  
    struct Node next; // ❌ ILLEGAL! infinite size  
};
```



```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *head = malloc(sizeof(struct Node));
    head->data = 10;
    head->next = malloc(sizeof(struct Node));
    head->next->data = 20;
    head->next->next = NULL;

    printf("First: %d\n", head->data);
    printf("Second: %d\n", head->next->data);

    // Free memory
    free(head->next);
    free(head);

    return 0;
}

```



Structures and Functions

1. How to Pass Elements of a Struct (individual members)

You can pass **individual fields** of a structure just like normal variables.

```
#include <stdio.h>

struct Book {
    char title[50];
    double price;
};

void printPrice(double p) {
    printf("Price: %.2lf\n", p);
}

int main() {
    struct Book b = {"C Programming", 199.99};
    printPrice(b.price); // Passing a member (element)
    return 0;
}
```



Structures and Functions

2. How to Pass a Struct Variable to a Function

You can pass the **entire struct** either:

- By **value** (copy)
- By **reference** (pointer)

A. Pass by Value (creates a copy)

```
void display(struct Book b) {  
    printf("Title: %s, Price: %.2lf\n", b.title, b.price);  
}
```

```
display(book); // pass whole struct
```

- *Changes inside display() won't affect the original.*



Structures and Functions

B. Pass by Reference (pointer to struct)

```
void modify(struct Book *b) {  
    b->price = 0.0;  
}  
  
modify(&book); // pass address of struct
```

- This allows the function to **modify** the original struct.
- This is the best approach due to the memory size optimization



Structures and Functions

3. How to Return a Struct from a Function

You can return a whole struct from a function **by value**.

Example:

```
1 struct Book createBook() {  
2     struct Book b;  
3     strcpy(b.title, "Clean Code");  
4     b.price = 150.75;  
5     return b;  
6 }  
7  
8 // in main  
9 struct Book myBook = createBook();
```

- Safe in modern compilers due to return value optimization (RVO). No need to use static variables unless required.
- **Return Value Optimization (RVO)** is a compiler optimization that **eliminates the need to copy a struct or object** when it is returned from a function.



- **Without RVO (conceptually):**

Traditionally, returning `b` would mean copying its entire contents to a new memory location in the caller. If `struct Book` is large, this could be **inefficient**.

- **With RVO (modern compilers):**

The compiler **detects** that `b` is just being returned, so instead of creating a local `b` and copying it, it directly **constructs `b` in the caller's memory space**. **No copy actually happens**. It's optimized out.

No need to make `b` `static` or allocate it with `malloc()` to "avoid returning a local variable."

```
1 struct Book createBook() {  
2     struct Book b;  
3     // fill b  
4     return b; // copy happens here (in theory)  
5 }
```



Old Practice (No longer recommended):

In older C code, you might see this pattern:

```
struct Book* getBook() {  
    static struct Book b; // stays alive after function ends  
  
    // fill b  
    return &b;  
}
```

This works, but:

- Not thread-safe
- Data is shared globally — can be **overwritten** if function is called again
- Avoids returning local variables — which is now unnecessary due to RVO

What Not to Do:

```
struct Book* wrongFunc() {  
    struct Book b;  
    // ...  
    return &b; // Returning address of a local variable (undefined behavior)  
}
```



Structures and Functions

4. How to Return a Pointer to a Struct

You can return a pointer to a dynamically allocated struct.

Example (with **malloc** - **static**):

```
1 struct Book* createBook() {  
2     struct Book *b = malloc(sizeof(struct Book));  
3     strcpy(b->title, "Effective C");  
4     b->price = 220.00;  
5     return b;  
6 }  
7  
8 struct Book *p = createBook();  
9 printf("%s - %.2lf\n", p->title, p->price);  
10 free(p); // Don't forget to free memory
```



```
1 #include <stdio.h>
2 #include <string.h>
3
4 struct Book {
5     char title[50];
6     double price;
7 };
8
9 // Function returns a pointer to a static struct
10 struct Book* getBook() {
11     static struct Book b;
12
13     strcpy(b.title, "C Programming");
14     b.price = 199.99;
15
16     return &b;
17 }
18
19 int main() {
20     struct Book *ptr = getBook();
21
22     printf("Title: %s\n", ptr->title);
23     printf("Price: %.2lf\n", ptr->price);
24
25     return 0;
26 }
```

Output

Title: C Programming

Price: 199.99

=== Code Execution Successful ===



Structures and Functions

In **C**, you **cannot** define actual *functions* inside a struct. However, you **can store pointers to functions** inside a structure.

```
#include <stdio.h>

// Define a function
void printDetails(const char *name, int age) {
    printf("Name: %s, Age: %d\n", name, age);
}

// Define a struct with a function pointer
struct Person {
    char name[50];
    int age;
    void (*print)(const char *, int); // function pointer
};

int main() {
    struct Person p = {"Alice", 30, printDetails};

    // Call the function through the struct
    p.print(p.name, p.age);

    return 0;
}
```



Structures and Functions

```
#include<stdio.h>

struct Operation {
    char symbol;
    int (*operate)(int, int);
};

int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

int main() {
    struct Operation ops[2] = {
        {'+', add},
        {'*', mul}
    };

    printf("3 + 4 = %d\n", ops[0].operate(3, 4));
    printf("3 * 4 = %d\n", ops[1].operate(3, 4));

    return 0;
}
```



Bit Fields

Bitfields in C allow you to define structure members that use **a specific number of bits**, rather than the default size of a full data type (like 4 bytes for an `int`). This is useful when you need to **save memory** or work with **binary data** (e.g., hardware registers, flags, etc.).

Syntax

```
struct {  
    data_type elem : width;  
};
```

```
struct Flags {  
    unsigned int isVisible : 1; // only 1 bit  
    unsigned int isEnabled : 1; // 1 bit  
    unsigned int mode      : 2; // 2 bits  
};
```

This uses only 4 bits total (instead of 3 full `int` fields = 96 bits!)



Example

```
1  #include<stdio.h>
2  struct Flags {
3      unsigned int isVisible : 1; // only 1 bit
4      unsigned int isEnabled : 1; // 1 bit
5      unsigned int mode      : 2; // 2 bits
6  };
7
8  struct Flags_1 {
9      unsigned int isVisible ;
10     unsigned int isEnabled ;
11     unsigned int mode      ;
12 };
13
14 int main(){
15     struct Flags f={1,0,2};
16     struct Flags_1 f1={1,0,2};
17
18     printf("Flags Size: %i Bytes ---- Flags_1: %i Bytes\n",sizeof(f),sizeof(f1));
19 }
```

Output

Flags Size: 4 Bytes ---- Flags_1: 12 Bytes

=== Code Execution Successful ===



Example

```
#include <stdio.h>

struct {
    unsigned int age : 3;
} Age;

int main() {
    Age.age = 4;
    printf("Sizeof(Age): %d\n", sizeof(Age)); // Size of struct in bytes
    printf("Age.age: %d\n", Age.age);         // Outputs 4

    Age.age = 7;
    printf("Age.age : %d\n", Age.age);         // Outputs 7 (max value for 3 bits)

    Age.age = 8;
    printf("Age.age : %d\n", Age.age);         // Undefined behavior (value overflow)

    return 0;
}
```

```
Sizeof(Age): 4
Age.age: 4
Age.age : 7
Age.age : 0
```

Overflow (8) 1000 to (0) 000 (truncated)



Union

A union is a special data type available in C that allows to store different data types in the same memory location. You can define a union with many members, but only one member can contain a value at any given time. Unions provide an efficient way of using the same memory location for multiple purpose.

```
union Data {  
    int i;        // 4 bytes  
    float f;      // 4 bytes  
    char str[20]; // 20 bytes  
};
```

Then, the size of union `Data` is **20 bytes** (the size of the largest member), and **all members start at the same memory location**.

Here, **Data** can hold:

- an **int**, or
- a **float**, or
- a **char[20]** (but **only one at a time**).



Union Example

Imagine we want to build a **stack of mixed values** (like Python or JavaScript allows).

Each item on the stack can be an `int`, `float`, `string`, or even a pointer — so we wrap it in this `Value` structure.

```
// 1. Define an enum for possible value types
- typedef enum {
    INTEGER,
    STRING,
    REAL,
    POINTER
} Type;

// 2. Define a struct that can hold any of these types
- typedef struct {
    Type type; // tells us which type is currently stored
- union {
    int integer; // if type == INTEGER
    char *string; // if type == STRING
    float real; // if type == REAL
    void *pointer; // if type == POINTER
    } x; // union named 'x'
} Value;
```



```

#include <stdio.h>
#include <string.h>

typedef enum {
    INTEGER,
    STRING,
    REAL,
    POINTER
} Type;

typedef struct {
    Type type;
    union {
        int integer;
        char *string;
        float real;
        void *pointer;
    } x;
} Value;

void printValue(Value v) {
    switch (v.type) {
        case INTEGER:
            printf("Integer: %d\n", v.x.integer);
            break;
        case STRING:
            printf("String: %s\n", v.x.string);
            break;
        case REAL:
            printf("Real: %.2f\n", v.x.real);
            break;
        case POINTER:
            printf("Pointer: %p\n", v.x.pointer);
            break;
    }
}

```

```

int main() {
    Value a, b, c;

    a.type = INTEGER;
    a.x.integer = 42;

    b.type = STRING;
    b.x.string = "Hello, union!";

    c.type = REAL;
    c.x.real = 3.1415;

    printValue(a);
    printValue(b);
    printValue(c);

    return 0;
}

```

Output

```

Integer: 42
String: Hello, union!
Real: 3.14

```

=== Code Execution Successful ===



We're creating a **simple dynamic type**. The `Value` struct is like a **"variant"**:

- You don't know what type it holds until you check the `type` field.
- The `union` saves memory (only the biggest member is allocated).
- This is common in:
 - **Virtual machines**
 - **Interpreters**
 - **Dynamic scripting engines**
 - **Serialization systems**



Anonymous Structures and Unions

An **anonymous struct or union** in C is a struct or union **without a name (tag)**. It allows its members to be **accessed directly** from the enclosing structure **without needing to refer to the inner struct/union name**.

```
#include <stdio.h>

struct Data {
    int type;

    union { // no union tag
        int i;
        float f;
        char str[20];
    }; // no name – anonymous union
};

int main() {
    struct Data d;

    d.type = 1;
    d.i = 42; // directly accessing union member

    printf("Type: %d, Integer: %d\n", d.type, d.i);

    return 0;
}
```

You can access `d.i`, `d.f`, or `d.str` **directly**, without writing `d.union_name.i`.



Anonymous struct example:

You access `emp.first` and `emp.last` **as if** they were direct members of `Employee`.

```
#include <stdio.h>
#include <string.h>
- struct Employee {
    int id;

-     struct { // without tag
        char first[20];
        char last[20];
    }; // anonymous struct without name
};

- int main() {
    struct Employee emp;

    emp.id = 101;
    strcpy(emp.first, "John");
    strcpy(emp.last, "Doe");

    printf("ID: %d, Name: %s %s\n", emp.id, emp.first, emp.last);

    return 0;
}
```



Enumeration

C enumeration (enum) is an enumerated data type that consists of a group of integral constants. **Enums** are useful when you want to assign user-defined names to integral constants. The **enum** keyword is used to define enums.

```
enum enum_name{const1, const2, ... };
```

```
enum Color { RED, GREEN, BLUE };
```

This defines a new type called enum Color with **three values**:

- RED = 0
- GREEN = 1
- BLUE = 2

Unless you specify otherwise, the first name gets the value 0, and the rest increase by 1.



```
#include <stdio.h>

enum Day { SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY };

int main() {
    enum Day today = WEDNESDAY;

    if (today == WEDNESDAY) {
        printf("It's midweek.\n");
    }

    printf("Day index is: %d\n", today); // prints: 3

    return 0;
}
```



Custom Values Enumeration

```
enum HttpStatus {  
    OK = 200,  
    NOT_FOUND = 404,  
    SERVER_ERROR = 500  
};
```

typedef enum: To avoid writing enum every time

```
typedef enum {  
    LOW,  
    MEDIUM,  
    HIGH  
} Priority;  
  
Priority p = HIGH;
```



Behind the Scenes

- An enum is stored as an `int` (usually 4 bytes).
- You can use `%d` in `printf` to print enum values.
- But they are **not strongly type-safe** in C (unlike in C++).

Common Use Cases

- States (e.g., `IDLE`, `RUNNING`, `PAUSED`, `STOPPED`)
- Options (e.g., `ON`, `OFF`)
- Flags and status codes
- Switch-case logic

Limitations

- Enums are **integers only**.
- You can assign any `int` to an enum variable (not strongly enforced).
- No way to print enum names directly — you'd have to write a function or use `switch`.



Lab 5

- Create a structure called "Student" with members name, age, and total marks. Write a C program to input data for the students, display their information, and find the average of total marks, the higher marks, the top 3 ranked.
- Create a structure named Complex to represent a complex number with real and imaginary parts. Write a C program to add and multiply two complex numbers.
- Library Management System, define a struct Book with fields: title, author, price, category, availability
 - Use an enum for: BookStatus, BookCategory
 - Allow operations:
 - Add a new book
 - Search by title or author
 - Change status (e.g., issue/return)
 - Count books in each category
- Employee Management System, define struct for employee records, enum for roles or departments and union for storing either hourly rate or monthly salary (based on employee type)
 - Define struct Employee with fields: id, name, department, type, salary/hourly
 - Use enum EmployeeType { FULL_TIME, PART_TIME, INTERN }
 - Use union Compensation to store either hourly_rate or monthly_salary
 - Tasks:
 - Add/remove employees
 - Filter by department or type
 - Calculate total payroll
 - Search employee by ID



Preprocessing Directives in C

In C, **preprocessing directives** are instructions that are handled **before the actual compilation** of code begins. They are processed by the **C preprocessor**.

All preprocessor directives start with the **# (hash)** symbol and **do not end with a semicolon**.

Purpose of Preprocessing Directives:

- Include code from other files
- Define constants and macros
- Conditional compilation
- Avoid multiple inclusions of the same file
- Control compilation flow



Directive	Description	Syntax	Example / Use Case
#define	Defines a macro or constant that replaces values before compilation.	#define name value	#define PI 3.14 #define MAX(a,b) ((a)>(b)?(a):(b))
#undef	Undefines a previously defined macro.	#undef name	#undef PI
#include	Inserts the contents of another file into the source code.	#include <filename> #include "filename"	#include <stdio.h> #include "myfile.h"
#ifdef	Compiles code only if the macro is defined.	#ifdef MACRO_NAME	#ifdef DEBUG
#ifndef	Compiles code only if the macro is not defined.	#ifndef MACRO_NAME	Common for header guards: #ifndef FILE_H #define FILE_H ... #endif
#if	Compiles code if the expression evaluates to true (non-zero) .	#if expression	#if VERSION == 2



Directive	Description	Syntax	Example / Use Case
#else	Provides alternative code if #if , #ifdef , or #ifndef fails.	#else	#else block
#elif	Else-if for preprocessor conditionals.	#elif expression	#elif VERSION == 3
#endif	Ends #if , #ifdef , or #ifndef block.	#endif	Closes the block
#error	Displays a compilation error with a custom message.	#error message	#error "Unsupported compiler version"
#line	Changes the compiler's idea of current line number and filename.	#line number "filename"	#line 100 "custom.c"
#pragma once	Ensures the file is included only once — safer and simpler than guards.	#pragma once	At the top of a header file
#pragma message	Displays a custom message during compilation — useful for reminders/debugging.	#pragma message("messagetext")	#pragma message("Compiling debug version")



Macros

Macros in C are the names given to specific constant values or code statements which are replaced with their value/code before the compilation processor. **C Macros** are defined using the **#define** preprocessor directive.

Types of C macros

- Object-like Macros
- Function-like Macros
- Chained Macros
- Variadic Macros
- Predefined Macros



1. Object-like Macros

```
#define name value

#define PI 3.14159

int main() {
    float area = PI * 2 * 2;
    return 0;
}
```

2. Function-like Macros

```
#define SQUARE(x) ((x) * (x))

int main() {
    int result = SQUARE(5); // becomes ((5) * (5))
    return 0;
}
```

```
#include <stdio.h>

#define LOOP(x) {\
    for (int i = 1; i <= x; i++)\
        printf("Iteration no: %d\n", i);\
}

int main() {

    int x = 3;
    LOOP(x);

}
```



Macro Function

A **macro function** is defined using `#define` and behaves like a function, but it performs **textual substitution before** compilation.

```
#define MACRO_NAME(parameters) expression
```

```
#include <stdio.h>

#define SQUARE(x) ((x) * (x)) // Macro function

int main() {
    int a = 5;
    printf("Square of %d = %d\n", a, SQUARE(a));
    return 0;
}
```

Limitation	Why
No type checking	Preprocessor doesn't check types
Hard to debug	Errors point to expanded code
Risk of multiple evaluation	If you pass expressions with side effects
Can cause bugs without parentheses	Always use extra parentheses



3. Chained Macros

```
#define LENGTH 10
#define WIDTH 5
#define AREA (LENGTH * WIDTH)

int area = AREA; // becomes (10 * 5)
```

4. Variadic Macros

```
#include <stdio.h>

#define LOG(format, ...) printf("LOG: " format "\n", __VA_ARGS__)

int main() {
    LOG("Value = %d", 55); // Expands to printf(...)
    return 0;
}
```



5. Predefined Macros

ANSI C defines a number of macros. Although each one is available for use in programming, the predefined macros should not be directly modified.

Macro	Description
<code>__DATE__</code>	The current date as a character literal in "MMM DD YYYY" format.
<code>__TIME__</code>	The current time as a character literal in "HH:MM:SS" format.
<code>__FILE__</code>	This contains the current filename as a string literal.
<code>__LINE__</code>	This contains the current line number as a decimal constant.
<code>__STDC__</code>	Defined as 1 when the compiler complies with the ANSI standard.

```
#include <stdio.h>

int main() {

    printf("File :%s\n", __FILE__ );
    printf("Date :%s\n", __DATE__ );
    printf("Time :%s\n", __TIME__ );
    printf("Line :%d\n", __LINE__ );
    printf("ANSI :%d\n", __STDC__ );

}
```



Examples

```
#include <stdio.h>
```

```
#define PI 3.14159
```

```
#define SQUARE(x) ((x) * (x))
```

```
int main() {  
    printf("PI = %.2f\n", PI);  
    printf("Square of 5 = %d\n", SQUARE(5));  
    return 0;  
}
```

```
#include <stdio.h>
```

```
#define TEMP 100
```

```
int main() {  
    printf("Before undef: %d\n", TEMP);
```

```
#undef TEMP
```

```
// TEMP is now undefined - using it will cause a compiler error  
// printf("%d\n", TEMP); // Uncommenting this will cause error  
    return 0;  
}
```

```
#include <stdio.h> // include header file
```

```
#define DEBUG // Comment this line to disable debug code
```

```
int main() {  
    // Conditional compilation - compile time switch  
    #ifdef DEBUG  
        printf("Debug mode is ON\n");  
    #endif  
  
    #ifndef RELEASE  
        printf("Release mode is OFF\n");  
    #endif  
  
    return 0;  
}
```

```
#include <stdio.h>
```

```
#define VERSION 2
```

```
int main() {  
    // Compile time switch  
    #if VERSION == 1  
        printf("Version 1 selected\n");  
    #elif VERSION == 2  
        printf("Version 2 selected\n");  
    #else  
        printf("Unknown version\n");  
    #endif  
  
    return 0;  
}
```

```
#ifndef MAX_SIZE
```

```
    #error "MAX_SIZE not defined"
```

```
#endif
```

```
int main() {  
    return 0;  
}
```



#pragma Directive

The preprocessor directive `#pragma` is used to provide additional information to the compiler in C/C++ language. This is used by the compiler to provide some special features.

The pragmas are compiler dependent. Not all the pragma directives are supported by all the compilers.



Do not use



Directive	Description
#pragma startup	Runs a specific function before <code>main()</code> begins execution. <i>(Compiler-specific; works in some compilers like Turbo C/C++)</i>
#pragma exit	Runs a specific function just before program exits . <i>(Also compiler-specific and may not work in modern GCC/Clang)</i>
#pragma warn	Used to suppress or enable specific warnings . <i>(Mostly supported in older compilers like Turbo C++)</i>
#pragma GCC dependency	Tells the compiler to check if another file is newer than the current file. If it is, a warning is issued.
#pragma GCC system_header	Treats the rest of the file as a system header — compiler suppresses warnings from it.
#pragma GCC poison	Prevents the use of specific identifiers. If you use a "poisoned" name, the compiler throws an error . Useful for deprecating code.
#pragma once	Ensures that the current header file is included only once , avoiding multiple definition issues. Cleaner alternative to include guards.



Examples

#pragma once — Prevent multiple inclusion

```
// myheader.h
#pragma once

void hello();
```

#pragma GCC poison — Block usage of deprecated identifiers

```
#pragma GCC poison gets

int main() {
    char s[50];
    //gets(s); // This will cause a compiler error if uncommented
    return 0;
}
```



Notes on #pragma startup and #pragma exit

These are supported in **Turbo C/C++** but **not in modern GCC/Clang**.

```
// Turbo C/C++ example
void before_main();
void after_main();

#pragma startup before_main
#pragma exit after_main

void before_main() {
    printf("Run before main\n");
}

void after_main() {
    printf("Run after program ends\n");
}

int main() {
    printf("Inside main\n");
    return 0;
}
```



Preprocessor operators

Operator	Name	Purpose
\ (backslash)	Continuation operator	Continues a macro definition across multiple lines
#	Stringizing operator	Converts a macro parameter to a string
##	Token-pasting (concatenation)	Joins two tokens into one during macro expansion
defined	Defined operator	Checks if a macro is defined (used with #if , #elif , etc.)



1. Continuation Operator \

Used to **split a macro** over multiple lines for better readability. The `\` tells the preprocessor that the macro continues to the next line.

```
#include <stdio.h>

#define LOG(msg) \
    printf("LOG: %s\n", msg);

int main() {
    LOG("This is a long log message.");
    return 0;
}
```

2. Stringizing Operator

Converts the macro argument into a string literal. `#x` turns the argument Hello World into "Hello World".

```
#include <stdio.h>

#define TO_STRING(x) #x

int main() {
    printf("%s\n", TO_STRING>Hello World)); // Output: Hello World (as a string)
}
```



3. Token-Pasting Operator

Joins two macro arguments into a **single token** (e.g., variable name, identifier).

merges `var_` and `id` to create the variable `var_id`.

```
#include <stdio.h>

#define CREATE_VAR(name) int var_##name = 100;

int main() {
    CREATE_VAR(id)    // becomes: int var_id = 100;
    printf("%d\n", var_id);
    return 0;
}
```



4. **defined** Operator

Used in conditional compilation to **check if a macro is defined**.

`defined(DEBUG)` evaluates to `true` because `DEBUG` was defined earlier.

```
#include <stdio.h>

#define DEBUG

int main() {
    #if defined(DEBUG)
        printf("Debug mode is ON\n");
    #else
        printf("Debug mode is OFF\n");
    #endif
    return 0;
}
```







Resources :

- ITI - Embedded System Track Materials
- C How to Program
- Tutorialspoint
- W3 Schools
- ANSI C



© ITI - Embedded System Track 2027. All rights reserved.

This course material, including all slides and content, is the intellectual property of the ITI - Embedded Systems Track.

